

Problem #4 Due: Friday, November 22, before midnight. Turn in to canvas, as described in the syllabus.

In this problem set, we will explore a classical chaotic dynamical system that has some relevance for space mission design. An elementary model for the orbits of complex space missions is the circular restricted 3-body problem (CR3BP). This considers the motion of a point mass m_3 in the gravitational fields of two more massive points m_2 and m_1 , with $m_1 > m_2$. The main reference, if you are interested in this topic, is the book by Koon, Lo, Marsden and Ross, which can be found here.

First, the ODEs. We will consider a rotating frame of reference such that m_1 and m_2 , which are moving about their center of mass, are fixed. In this frame of reference, we consider x, y, z to be the three dimensional position of m_3 (the spacecraft) and write down Euler-Lagrange equations (Koon et al Chapter 2):

$$\begin{aligned}\frac{d^2x}{dt^2} &= 2\frac{dy}{dt} - \frac{\partial U}{\partial x} \\ \frac{d^2y}{dt^2} &= -2\frac{dx}{dt} - \frac{\partial U}{\partial y} \\ \frac{d^2z}{dt^2} &= -\frac{\partial U}{\partial z}\end{aligned}\tag{1}$$

where the effective potential, $U(x, y, z)$ is given by

$$U(x, y, z) = -\frac{1}{2}(x^2 + y^2) - \frac{\mu_1}{r_1(x, y, z)} - \frac{\mu_2}{r_2(x, y, z)} - \frac{1}{2}\mu_1\mu_2.\tag{2}$$

Here, $r_1^2 = (x + \mu_2)^2 + y^2 + z^2$ and $r_2^2 = (x - \mu_1)^2 + y^2 + z^2$ are squared distances from m_1 , which is located at $(-\mu_2, 0, 0)$, and m_2 , which is located at $(\mu_1, 0, 0)$, respectively.

Consider a 6-dimensional phase space with variables $X = (x, y, z, dx/dt, dy/dt, dz/dt)$ and the flow described by (1), which will reduce to a first order ODE system for $X : \partial\varphi^t(X)/dt = V(\varphi^t(X))$.

Let us consider $\mu_2 = 0.1$ and $\mu_1 = 1 - \mu_2$, which could be the values when m_1 is Pluto and m_2 is one of its moons.

1. Derive the vector field V , which should give you a nonlinear autonomous dynamical system in $\mathbb{R}^6$. Plot the potential function U on the $x-y$ plane. How many critical points do you see? (3 points)

We start from the given system of second-order equations for the (CR3BP):

$$\begin{aligned}\frac{d^2x}{dt^2} &= 2 \frac{dy}{dt} - \frac{\partial U}{\partial x}, \\ \frac{d^2y}{dt^2} &= -2 \frac{dx}{dt} - \frac{\partial U}{\partial y}, \\ \frac{d^2z}{dt^2} &= -\frac{\partial U}{\partial z},\end{aligned}$$

where the effective potential $U(x, y, z)$ is

$$U(x, y, z) = -\frac{1}{2}(x^2 + y^2) - \frac{\mu_1}{r_1(x, y, z)} - \frac{\mu_2}{r_2(x, y, z)} - \frac{1}{2}\mu_1\mu_2.$$

Here

$$\begin{aligned}r_1^2(x, y, z) &= (x + \mu_2)^2 + y^2 + z^2, \\ r_2^2(x, y, z) &= (x - \mu_1)^2 + y^2 + z^2,\end{aligned}$$

so that $r_1 = \sqrt{(x + \mu_2)^2 + y^2 + z^2}$ and $r_2 = \sqrt{(x - \mu_1)^2 + y^2 + z^2}$ are the distances from the third body to the masses m_1 and m_2 , located at $(-\mu_2, 0, 0)$ and $(\mu_1, 0, 0)$ respectively. We take

$$\mu_2 = 0.1, \quad \mu_1 = 1 - \mu_2 = 0.9.$$

We introduce velocity variables $u = \frac{dx}{dt}, v = \frac{dy}{dt}, w = \frac{dz}{dt}$ and collect the phase-space variables into the vector $X = (x, y, z, u, v, w) \in \mathbb{R}^6$. Then we can write the system as a first-order system

$$\frac{dX}{dt} = V(X),$$

for some vector field $V : \mathbb{R}^6 \rightarrow \mathbb{R}^6$. To find V , firstly notice that the first three components are simply the definitions of the velocities:

$$\frac{dx}{dt} = u, \quad \frac{dy}{dt} = v, \quad \frac{dz}{dt} = w.$$

For the remaining three, we start by computing the partial derivatives of U . W.r.t. x , the derivatives of the different terms are

$$\begin{aligned}\frac{\partial}{\partial x} \left[-\frac{1}{2}(x^2 + y^2) \right] &= -x \\ \frac{\partial}{\partial x} \left(-\frac{\mu_1}{r_1} \right) &= -\mu_1 \frac{\partial}{\partial x} (r_1^{-1}) = -\mu_1 (-1) r_1^{-2} \frac{\partial r_1}{\partial x} = \mu_1 r_1^{-2} \cdot \frac{x + \mu_2}{r_1} = \mu_1 \frac{x + \mu_2}{r_1^3} \\ \frac{\partial}{\partial x} \left(-\frac{\mu_2}{r_2} \right) &= \mu_2 \frac{x - \mu_1}{r_2^3}.\end{aligned}$$

W.r.t. y , the derivatives of the different terms are

$$\frac{\partial}{\partial y} \left[-\frac{1}{2}(x^2 + y^2) \right] = -y.$$

and with

$$\frac{\partial r_1}{\partial y} = \frac{y}{r_1}, \quad \frac{\partial r_2}{\partial y} = \frac{y}{r_2}$$

we have

$$\frac{\partial}{\partial y} \left(-\frac{\mu_1}{r_1} \right) = \mu_1 \frac{y}{r_1^3}, \quad \frac{\partial}{\partial y} \left(-\frac{\mu_2}{r_2} \right) = \mu_2 \frac{y}{r_2^3}.$$

W.r.t. z , the derivatives of the terms are 0 for $-\frac{1}{2}(x^2 + y^2)$ and, with

$$\frac{\partial r_1}{\partial z} = \frac{z}{r_1} \text{ and } \frac{\partial r_2}{\partial z} = \frac{z}{r_2},$$

$$\frac{\partial}{\partial z} \left(-\frac{\mu_1}{r_1} \right) = \mu_1 \frac{z}{r_1^3} \text{ and } \frac{\partial}{\partial z} \left(-\frac{\mu_2}{r_2} \right) = \mu_2 \frac{z}{r_2^3}.$$

Overall, we have:

$$\begin{aligned} \frac{d^2x}{dt^2} &= 2\frac{dy}{dt} - \frac{\partial U}{\partial x} = 2v - \left(-x + \mu_1 \frac{x + \mu_2}{r_1^3} + \mu_2 \frac{x - \mu_1}{r_2^3} \right) \\ &= 2v + x - \mu_1 \frac{x + \mu_2}{r_1^3} - \mu_2 \frac{x - \mu_1}{r_2^3}, \end{aligned}$$

$$\begin{aligned} \frac{d^2y}{dt^2} &= -2\frac{dx}{dt} - \frac{\partial U}{\partial y} = -2u - \left(-y + \mu_1 \frac{y}{r_1^3} + \mu_2 \frac{y}{r_2^3} \right) \\ &= -2u + y - \mu_1 \frac{y}{r_1^3} - \mu_2 \frac{y}{r_2^3}, \end{aligned}$$

$$\frac{d^2z}{dt^2} = -\frac{\partial U}{\partial z} = -\mu_1 \frac{z}{r_1^3} - \mu_2 \frac{z}{r_2^3}.$$

Using $u = dx/dt$, $v = dy/dt$, $w = dz/dt$, the second derivatives become

du/dt , dv/dt , dw/dt , so the full first-order system in $\mathbb{R}^6$ is

$$\begin{cases} \frac{dx}{dt} = u, \\ \frac{dy}{dt} = v, \\ \frac{dz}{dt} = w, \\ \frac{du}{dt} = 2v + x - \mu_1 \frac{x + \mu_2}{r_1^3} - \mu_2 \frac{x - \mu_1}{r_2^3}, \\ \frac{dv}{dt} = -2u + y - \mu_1 \frac{y}{r_1^3} - \mu_2 \frac{y}{r_2^3}, \\ \frac{dw}{dt} = -\mu_1 \frac{z}{r_1^3} - \mu_2 \frac{z}{r_2^3}, \end{cases}$$

where

$$r_1^2 = (x + \mu_2)^2 + y^2 + z^2, \quad r_2^2 = (x - \mu_1)^2 + y^2 + z^2.$$

So the vector field $V : \mathbb{R}^6 \rightarrow \mathbb{R}^6$ is

$$V(x, y, z, u, v, w) = \begin{pmatrix} u \\ v \\ w \\ 2v + x - \mu_1 \frac{x + \mu_2}{r_1^3} - \mu_2 \frac{x - \mu_1}{r_2^3} \\ -2u + y - \mu_1 \frac{y}{r_1^3} - \mu_2 \frac{y}{r_2^3} \\ -\mu_1 \frac{z}{r_1^3} - \mu_2 \frac{z}{r_2^3} \end{pmatrix}.$$

This is a nonlinear autonomous (time-independent) dynamical system in $\mathbb{R}^6$.

To visualize the potential on the x - y plane, we restrict to $z = 0$:

$$U(x, y, 0) = -\frac{1}{2}(x^2 + y^2) - \frac{\mu_1}{\sqrt{(x + \mu_2)^2 + y^2}} - \frac{\mu_2}{\sqrt{(x - \mu_1)^2 + y^2}} - \frac{1}{2}\mu_1\mu_2.$$

For the given values $\mu_1 = 0.9$, $\mu_2 = 0.1$, this becomes

$$U(x, y, 0) = -\frac{1}{2}(x^2 + y^2) - \frac{0.9}{\sqrt{(x + 0.1)^2 + y^2}} - \frac{0.1}{\sqrt{(x - 0.9)^2 + y^2}} - \frac{1}{2}(0.9)(0.1).$$

Critical points of $U(x, y, 0)$ are solutions of

$$\nabla U(x, y, 0) = 0, \quad \text{i.e.} \quad \frac{\partial U}{\partial x}(x, y, 0) = 0, \quad \frac{\partial U}{\partial y}(x, y, 0) = 0.$$

These are precisely the equilibrium points of the CR3BP when the velocities (u, v, w) are zero.

When plotting $U(x, y, 0)$, we see five distinct critical points: three colinear points along the x -axis, lying between and outside the two masses (say L_1, L_2, L_3), and two additional critical points off the x -axis, forming equilateral triangles with the two masses (say L_4, L_5).

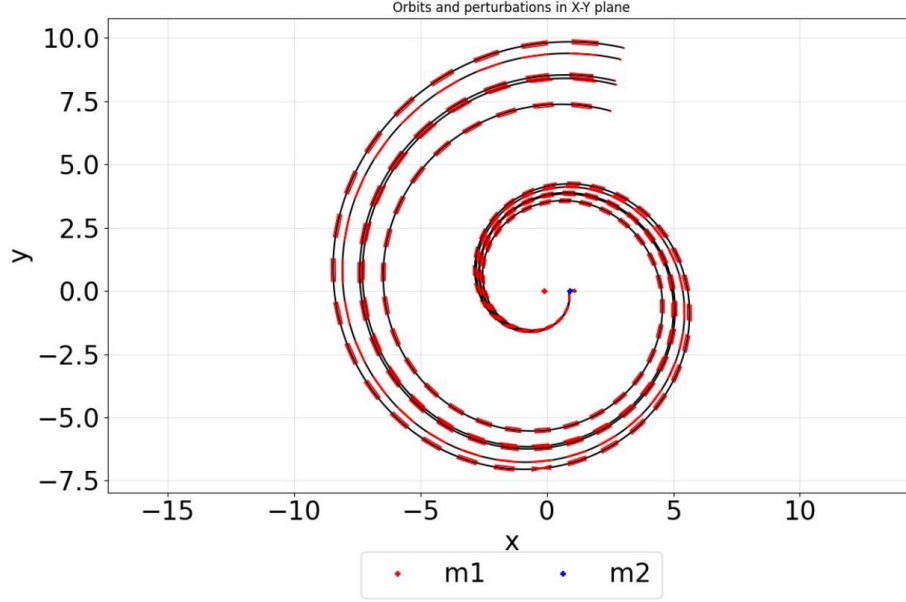


Figure 1: In black, we show orbits of the CR3BP on the x - y plane. The positions of the two masses are shown for $\mu_2 = 0.1$. In red, we show that leading Lyapunov vector computed numerically using power iteration. The initial and final points (at time 10) are shown in green and red respectively for 5 different orbits. The initial points are around L_2

2. We will now work out the physically meaningful fixed points of this system, which are the equilibrium points. That is, these are points where ∇U is the zero vector in $\mathbb{R}^3$, and the gravitation pull of the m_1 and m_2 are perfectly balanced. There are 5 such equilibrium points (see Figure 2.5.1 of Koon et al), three of which are on the x axis ($y = z = 0$). Find (approximately) the two equilibrium points closest to m_2 , on the x axis. (5 points)

Notice that if you used gradient-based approaches or Newton's method, you may miss these equilibrium points. Why is that? (2 points)

An equilibrium point (fixed point) in the full 6D phase space $X = (x, y, z, \dot{x}, \dot{y}, \dot{z})$ is a point where

$$\dot{x} = \dot{y} = \dot{z} = 0, \quad \ddot{x} = \ddot{y} = \ddot{z} = 0.$$

From above this is equivalent to $\nabla U(x, y, z) = 0$, so equilibrium points correspond exactly to critical points of the scalar potential U in $\mathbb{R}^3$. We want to find the two equilibrium points on the x -axis closest to m_2 , i.e. the two roots of $\nabla U = 0$ with $y = z = 0$ closest to $x = \mu_1 = 0.9$.

Restricting to the x -axis, $y = 0$ and $z = 0$, so

$$r_1 = \sqrt{(x + \mu_2)^2} = |x + \mu_2|, \quad r_2 = \sqrt{(x - \mu_1)^2} = |x - \mu_1|.$$

and

$$\frac{\partial U}{\partial y}(x, 0, 0) = 0, \quad \frac{\partial U}{\partial z}(x, 0, 0) = 0$$

for all x . Thus $\nabla U(x, 0, 0) = 0$ is equivalent to $\frac{\partial U}{\partial x}(x, 0, 0) = 0$.

So we want

$$\frac{\partial U}{\partial x}(x, 0, 0) = -x + \mu_1 \frac{x + \mu_2}{|x + \mu_2|^3} + \mu_2 \frac{x - \mu_1}{|x - \mu_1|^3} = 0.$$

subject also to $y = 0, z = 0$

Because of the singularities at $x = -\mu_2 = -0.1$ and $x = \mu_1 = 0.9$ (where $U \rightarrow -\infty$), the equation is nonlinear and cannot be solved in closed form. We therefore solve it numerically (see code)

Now, if we used gradient-based approaches or the naive Newton's method, we may fail to find these equilibrium points. The reasons are two-fold. Firstly, these points are saddles, not minima of U . The function $U(x, y, z)$ has singularities at the locations of the primaries:

$$U(x, y, z) \rightarrow -\infty \quad \text{as} \quad (x, y, z) \rightarrow (-\mu_2, 0, 0) \text{ or } (\mu_1, 0, 0)$$

so U has very deep potential wells at the masses. The colinear equilibria L_1 and L_2 lie between and outside these wells and are in fact saddle points of U (they are unstable equilibria for the full dynamics).

A standard gradient-descent algorithm

$$X_{n+1} = X_n - \eta \nabla U(X_n),$$

with step size $\eta > 0$, always moves in the direction of decreasing U . Unless the initial point lies exactly on the stable manifold of a saddle point, gradient descent will flow toward a nearby local minimum (or, in this problem, toward the singular wells at the masses) rather than stopping at L_1 or L_2 . Thus gradient descent would almost never converge to these saddle equilibria.

The second reason is to solve $\nabla U(x) = 0$, Newton's method iterates

$$x_{n+1} = x_n - [\nabla^2 U(x_n)]^{-1} \nabla U(x_n).$$

but this is a local method: it converges rapidly only if the initial guess x_0 lies in the basin of attraction of the desired critical point. For the CR3BP potential: the Hessian $\nabla^2 U$ can be badly conditioned near the singularities at m_1 and m_2 , causing very large or erratic Newton steps that may "jump" away from L_1 or L_2 or even diverge. Moreover, since L_1 and L_2 are saddles, the Newton basins associated with them are typically small and interwoven with the basins of the other critical points (including the deep wells near the masses). Starting from a generic point in the plane, Newton's method is much more likely to converge to a nearby minimum (or fail to converge) than to find these saddle points.

These two fixed points, closest to m_2 are called, by convention, L_1 and L_2 . The ' L ' indicates that these are libration (oscillation) points, about which the spacecraft may move. Let the point L_2 have both m_1 and m_2 on the same side.

3. Start close to L_2 and generate orbits. In your answer, attach orbits from different initial conditions near L_2 on the $x - y$ plane. (2 points). Since the problem has been non-dimensionalized, time 2π , corresponds to one orbital rotation of m_1 and m_2 . You could generate the orbit up to this time or longer. Prove that there is a function (hint: this is the total energy) that remains constant along orbits (2 points).

(a) See code

(b) We define the energy function $E : \mathbb{R}^6 \rightarrow \mathbb{R}$ by

$$E(x, y, z, u, v, w) = \frac{1}{2} (u^2 + v^2 + w^2) + U(x, y, z).$$

The first term is the kinetic energy in the rotating frame, and the second term is the effective potential. We claim that E is constant along any solution $t \mapsto X(t)$ of our first-order system.

Let

$$X(t) = (x(t), y(t), z(t), u(t), v(t), w(t)),$$

be a solution. Along this orbit, we have

$$E(t) = \frac{1}{2} (u(t)^2 + v(t)^2 + w(t)^2) + U(x(t), y(t), z(t)).$$

and

$$\dot{E}(t) = u\dot{u} + v\dot{v} + w\dot{w} + U_x\dot{x} + U_y\dot{y} + U_z\dot{z},$$

Using the equations of motion from the system obtained in Question 1,

$$\dot{x} = u, \quad \dot{y} = v, \quad \dot{z} = w,$$

and

$$\dot{u} = 2v - U_x, \quad \dot{v} = -2u - U_y, \quad \dot{w} = -U_z.$$

Substituting these into $\dot{E}$:

$$\begin{aligned} \dot{E} &= u(2v - U_x) + v(-2u - U_y) + w(-U_z) + U_x u + U_y v + U_z w \\ &= (2uv - 2uv) + (-uU_x + U_x u) + (-vU_y + U_y v) + (-wU_z + U_z w) \\ &= 0, \end{aligned}$$

Such systems are called Hamiltonian systems. This is an example of a system where there is exactly one quantity that remains invariant!

4. Check that the energy is invariant along the orbits you generated. If it is not, explain how the choice of timestep and the time integration scheme affects the invariance (2 points). With Forward Euler, the orbits may look like the black lines in Figure 1.

We use two different one-step methods in our numerical analysis, forward Euler (FE):

$$X_{n+1} = X_n + \Delta t V(X_n),$$

which is a first-order (global error $O(\Delta t)$) and classical 4th-order Runge-Kutta (RK4):

$$\begin{aligned} k_1 &= V(X_n), \\ k_2 &= V\left(X_n + \frac{\Delta t}{2} k_1\right), \\ k_3 &= V\left(X_n + \frac{\Delta t}{2} k_2\right), \\ k_4 &= V(X_n + \Delta t k_3), \\ X_{n+1} &= X_n + \frac{\Delta t}{6} (k_1 + 2k_2 + 2k_3 + k_4), \end{aligned}$$

which has global error $O(\Delta t^4)$. For each method and each time step Δt we compute the orbit $\{X_n\}_{n=0}^N$ and evaluate

$$E_n = E(X_n), \quad \varepsilon_n = \frac{E_n - E_0}{|E_0|},$$

the energy and its relative error.

We perform this experiment for

$$\Delta t \in \{10^{-3}, 5 \times 10^{-3}\},$$

using both FE and RK4.

The numerical results printed by the code are (rounded to a few digits):

Method	Δt	$\max_n \varepsilon_n $	qualitative behavior
Forward Euler	10^{-3}	$\approx 1.17 \times 10^{-2}$	small but clear monotone drift
Forward Euler	5×10^{-3}	$\approx 5.95 \times 10^{-2}$	much larger drift, orbit deforms strongly
RK4	10^{-3}	$\approx 5.1 \times 10^{-13}$	energy essentially constant (roundoff level)
RK4	5×10^{-3}	$\approx 1.7 \times 10^{-11}$	still extremely small variation

Firstly, we discuss the role of the timestep Δt . Any one-step method of order p has a local truncation error of size $O(\Delta t^{p+1})$ and a global error of size $O(\Delta t^p)$ after integrating over a fixed time interval. This means that if Δt is small, each step moves the numerical solution only slightly off the true energy level, so E_n stays close to E_0 and the relative error

ε_n remains small, but if Δt is large, the per-step error is large, the orbit deviates significantly from the true trajectory, and the computed energy E_n exhibits large oscillations and/or systematic drift. Hence, decreasing the timestep improves the approximate invariance of E , but at the cost of more steps and higher computational cost.

However, the choice of time integrator matters even more: for Forward Euler with $\Delta t = 10^{-3}$ we have $E_0 \approx -1.77829$, $E_{\min} \approx -1.79909$, $E_{\max} \approx -1.77829$ so E decreases by about 1.2% over $[0, t_{\max}]$. With the coarser step $\Delta t = 5 \times 10^{-3}$ the maximum relative error grows to about 6%; the orbit spirals away from the "true" constant-energy trajectory and can look qualitatively like the black spirals in Figure 1.

By contrast, RK4 keeps the energy extremely close to its initial value: the relative error stays on the order of 10^{-12} - 10^{-11} even for $\Delta t = 5 \times 10^{-3}$. The remaining variation is dominated by roundoff error and the fact that RK4 is not exactly energy preserving.

Interpretation: forward Euler is only first order, so its global truncation error is $O(\Delta t)$, so the energy error accumulates almost monotonically, producing a noticeable drift. Increasing the time step by a factor of 5 increases the maximum relative energy error by roughly the same factor. RK4 has global error $O(\Delta t^4)$, so for the same Δt its energy error is many orders of magnitude smaller. The energy oscillates slightly around E_0 , but there is essentially no visible drift on the time scale of the experiment.

Note: there are so-called symplectic integrators (e.g. Stormer-Verlet / leapfrog, symplectic partitioned Runge-Kutta) which are specifically designed for Hamiltonian systems. They do not preserve the original Hamiltonian E exactly, but they preserve a nearby "modified" Hamiltonian. As a result, the numerical energy error remains bounded and typically oscillatory over very long times, and the orbits stay close to the true energy surface.

5. First derive the linearized equations around any fixed orbit, say, $X(t)$. These should be given by

$$\frac{\partial u(t)}{\partial t} = \nabla V(X(t))u(t), \quad (3)$$

where $u(t)$ is a tangent vector at $X(t)$. The function tangent in the file (crudely) approximates the solution of this equation. Using this function or writing your own, answer the following questions (1 point each):

- Is the orbit you picked chaotic? What object in the code tells you the answer?
- If you randomly picked an initial condition, say in a uniform interval on the x -axis around L_2 , would the generated orbit be chaotic?
- What are the other Lyapunov exponents computed by the code?
- What are the true values of the Lyapunov exponents of the system?
- Explain the sources of numerical error in the calculation of the LEs in the code.
- Does this linearized system follow Oseledets theorem?
- Is this system uniformly hyperbolic? Answer this question using the true values of the LEs.
- Do infinitesimal volumes in $\mathbb{R}^6$ contract or expand under the linearized dynamics? How do we know numerically?
- Do infinitesimal volumes in $\mathbb{R}^6$ contract or expand under the original dynamics (flow of V)? How do we know?

Hint: you may also find the function `plot_les` helpful

- (a) Fix a particular solution $X(t)$ of the nonlinear system

$$\dot{X}(t) = V(X(t)).$$

We now consider a one-parameter family of solutions $X(t; \varepsilon)$ depending smoothly on a small parameter ε , with

$$X(t; 0) = X(t), \quad X(0; \varepsilon) = X(0) + \varepsilon u_0 + O(\varepsilon^2),$$

for some initial perturbation $u_0 \in \mathbb{R}^6$. Each $X(t; \varepsilon)$ satisfies the ODE

$$\frac{\partial}{\partial t} X(t; \varepsilon) = V(X(t; \varepsilon)).$$

Define the perturbation vector

$$u(t) = \left. \frac{\partial}{\partial \varepsilon} X(t; \varepsilon) \right|_{\varepsilon=0}$$

We differentiate the ODE with respect to ε and then evaluate at $\varepsilon = 0$:

$$\frac{\partial}{\partial t} u(t) = \frac{\partial}{\partial \varepsilon} V(X(t; \varepsilon)) \Big|_{\varepsilon=0}.$$

By the chain rule,

$$\frac{\partial}{\partial \varepsilon} V(X(t; \varepsilon)) \Big|_{\varepsilon=0} = (DV)(X(t)) u(t) = \nabla V(X(t)) u(t),$$

where DV (or ∇V) is the 6×6 Jacobian matrix of partial derivatives of V .

Thus we obtain the linearized (variational) system along the reference orbit $X(t)$:

$$\frac{\partial u(t)}{\partial t} = \nabla V(X(t)) u(t).$$

In code, this Jacobian is computed by `jax.jacfwd(vectorField)` evaluated at `x0 = X(t)`.

We find the explicit form of the Jacobian. Write $X = (x, y, z, u, v, w)$, $u_{\text{lin}} = (\delta x, \delta y, \delta z, \delta u, \delta v, \delta w)^T$ so that $u(t)$ represents an infinitesimal perturbation in each coordinate. The Jacobian matrix $A(X) = \nabla V(X)$ has entries

$$A_{ij}(X) = \frac{\partial V_i}{\partial X_j}(X),$$

and we can write it in block form as

$$A(X) = \begin{pmatrix} 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 \\ * & * & * & 0 & 2 & 0 \\ * & * & * & -2 & 0 & 0 \\ * & * & * & 0 & 0 & 0 \end{pmatrix},$$

where the “*” entries involve second derivatives of the potential U , and the first three rows are obtained from

$$V_1 = \dot{x} = u, \quad V_2 = \dot{y} = v, \quad V_3 = \dot{z} = w,$$

so

$$\frac{\partial V_1}{\partial u} = 1, \quad \frac{\partial V_2}{\partial v} = 1, \quad \frac{\partial V_3}{\partial w} = 1,$$

and all other partial derivatives in those rows are zero.

The last three components of V are

$$V_4 = \dot{u} = 2v - U_x, \quad V_5 = \dot{v} = -2u - U_y, \quad V_6 = \dot{w} = -U_z,$$

so differentiating them gives

$$\begin{aligned}\frac{\partial V_4}{\partial x} &= -U_{xx}, & \frac{\partial V_4}{\partial y} &= -U_{xy}, & \frac{\partial V_4}{\partial z} &= -U_{xz}, & \frac{\partial V_4}{\partial u} &= 0, & \frac{\partial V_4}{\partial v} &= 2, & \frac{\partial V_4}{\partial w} &= 0, \\ \frac{\partial V_5}{\partial x} &= -U_{yx}, & \frac{\partial V_5}{\partial y} &= -U_{yy}, & \frac{\partial V_5}{\partial z} &= -U_{yz}, & \frac{\partial V_5}{\partial u} &= -2, & \frac{\partial V_5}{\partial v} &= 0, & \frac{\partial V_5}{\partial w} &= 0, \\ \frac{\partial V_6}{\partial x} &= -U_{zx}, & \frac{\partial V_6}{\partial y} &= -U_{zy}, & \frac{\partial V_6}{\partial z} &= -U_{zz}, & \frac{\partial V_6}{\partial u} &= 0, & \frac{\partial V_6}{\partial v} &= 0, & \frac{\partial V_6}{\partial w} &= 0,\end{aligned}$$

Recall

$$r_1^2 = (x + \mu_2)^2 + y^2 + z^2, \quad r_2^2 = (x - \mu_1)^2 + y^2 + z^2,$$

and denote

$$s_1 = x + \mu_2, \quad s_2 = x - \mu_1.$$

From the first-order derivatives,

$$U_x = -x + \mu_1 \frac{s_1}{r_1^3} + \mu_2 \frac{s_2}{r_2^3}, \quad U_y = -y + \mu_1 \frac{y}{r_1^3} + \mu_2 \frac{y}{r_2^3}, \quad U_z = \mu_1 \frac{z}{r_1^3} + \mu_2 \frac{z}{r_2^3},$$

we obtain:

$$\begin{aligned}U_{xx} &= \frac{\partial}{\partial x} U_x = -1 + \mu_1 \frac{\partial}{\partial x} \left(\frac{s_1}{r_1^3} \right) + \mu_2 \frac{\partial}{\partial x} \left(\frac{s_2}{r_2^3} \right) \\ &= -1 + \mu_1 \left(\frac{1}{r_1^3} - 3 \frac{s_1^2}{r_1^5} \right) + \mu_2 \left(\frac{1}{r_2^3} - 3 \frac{s_2^2}{r_2^5} \right) \\ U_{xy} &= \frac{\partial}{\partial y} U_x = \mu_1 \left(-3 \frac{s_1 y}{r_1^5} \right) + \mu_2 \left(-3 \frac{s_2 y}{r_2^5} \right) \\ U_{xz} &= \frac{\partial}{\partial z} U_x = \mu_1 \left(-3 \frac{s_1 z}{r_1^5} \right) + \mu_2 \left(-3 \frac{s_2 z}{r_2^5} \right) \\ U_{yy} &= \frac{\partial}{\partial y} U_y = -1 + \mu_1 \left(\frac{1}{r_1^3} - 3 \frac{y^2}{r_1^5} \right) + \mu_2 \left(\frac{1}{r_2^3} - 3 \frac{y^2}{r_2^5} \right) \\ U_{yz} &= \frac{\partial}{\partial z} U_y = \mu_1 \left(-3 \frac{yz}{r_1^5} \right) + \mu_2 \left(-3 \frac{yz}{r_2^5} \right) \\ U_{zz} &= \frac{\partial}{\partial z} U_z = \mu_1 \left(\frac{1}{r_1^3} - 3 \frac{z^2}{r_1^5} \right) + \mu_2 \left(\frac{1}{r_2^3} - 3 \frac{z^2}{r_2^5} \right)\end{aligned}$$

By symmetry of second derivatives:

$$U_{xy} = U_{yx}, \quad U_{xz} = U_{zx}, \quad U_{yz} = U_{zy}.$$

Collecting all this, the linearized dynamics for the perturbation $u_{\text{lin}} = (\delta x, \delta y, \delta z, \delta u, \delta v, \delta w)^T$ around the orbit $X(t)$ are

$$\frac{d}{dt} \begin{pmatrix} \delta x \\ \delta y \\ \delta z \\ \delta u \\ \delta v \\ \delta w \end{pmatrix} = A(X(t)) \begin{pmatrix} \delta x \\ \delta y \\ \delta z \\ \delta u \\ \delta v \\ \delta w \end{pmatrix},$$

where

$$A(X) = \begin{pmatrix} 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 \\ -U_{xx} & -U_{xy} & -U_{xz} & 0 & 2 & 0 \\ -U_{yx} & -U_{yy} & -U_{yz} & -2 & 0 & 0 \\ -U_{zx} & -U_{zy} & -U_{zz} & 0 & 0 & 0 \end{pmatrix}_{X=(x,y,z,u,v,w)}.$$

(b)

- Is the orbit you picked chaotic? What object in the code tells you the answer?

An orbit is chaotic if its largest Lyapunov exponent is strictly positive. In the code, this information is contained in the array `le` returned by `tangent`. Inspecting `plot_les(le)` one sees that, for each orbit, at least one of the six exponents lies well above zero (for most orbits, the top band is around $\lambda_1 \approx 0.6$ – 0.7). Therefore the orbits are chaotic.

- If you randomly picked an initial condition, say in a uniform interval on the x -axis around L_2 , would the generated orbit be chaotic?

The code chooses initial conditions in a small Gaussian neighbourhood of the nominal L_2 point and computes LEs for 100 such orbits (`batch_size = 100`). In the LE plot, for almost every orbit index on the horizontal axis, the top exponent is positive. Only a set of measure zero (those lying exactly on stable/center manifolds of special regular orbits) could have non-positive largest exponent.

Thus, if we randomly pick an x_0 in a small interval around L_2 on the x -axis and choose generic small velocities, the resulting orbit will almost surely be chaotic.

- What are the true values of the Lyapunov exponents of the system? For a smooth Hamiltonian flow $\dot{X} = J\nabla H(X)$ on $\mathbb{R}^{2n}$, we have the following two facts:

- (a) The flow preserves phase-space volume (Liouville's theorem). In terms of Lyapunov exponents this implies

$$\lambda_1 + \lambda_2 + \cdots + \lambda_{2n} = 0.$$

- (b) The derivative $D\varphi^t(X_0)$ is a symplectic linear map for each t . For a symplectic linear cocycle, the Lyapunov exponents come in $\pm$ pairs: if λ is an exponent, then so is $-\lambda$, with the same multiplicity.

In our case $n = 3$, so there are six exponents and they satisfy

$$\lambda_1 + \lambda_2 + \lambda_3 + \lambda_4 + \lambda_5 + \lambda_6 = 0,$$

and the multiset $\{\lambda_i\}$ is symmetric under $\lambda \mapsto -\lambda$.

The CR3BP in the rotating frame is an autonomous Hamiltonian system with one conserved quantity, the Jacobi integral

$$H(X) = \frac{1}{2}(u^2 + v^2 + w^2) + U(x, y, z),$$

so that $H(\varphi^t(X_0))$ is constant in t , so: (1) there is always a Lyapunov exponent equal to 0 associated with the direction of the flow itself. Perturbations tangent to the orbit merely reparametrize time along the same trajectory, so their average exponential growth rate is exactly zero. (2) Because of the symplectic $\pm$ -pairing and the presence of the first integral H , there is a second zero exponent: roughly speaking, perturbations which change the energy level but stay transverse to the flow also have zero exponential growth on average. (More formally, for a 3-d.o.f. Hamiltonian system with one integral, there are two independent neutral directions in tangent space: the flow direction and its symplectic conjugate.)

Therefore, among the six Lyapunov exponents we have at least

$$\lambda_3 = 0, \quad \lambda_4 = 0.$$

Putting together the fact that we have $\pm$ -pairing for Hamiltonian systems, $\sum_{i=1}^6 \lambda_i = 0$, and the fact that we have two exact zero exponents, we conclude that, for almost every orbit in a chaotic region of the CR3BP, the true Lyapunov spectrum must be of the form

$$\{\lambda_1, \lambda_2, 0, 0, -\lambda_2, -\lambda_1\},$$

with

$$\lambda_1 \geq \lambda_2 \geq 0.$$

Here λ_1 and λ_2 are nonnegative numbers that depend on the invariant set (energy level and region of phase space) and are typically strictly positive for chaotic orbits near L_2 , while they vanish for regular quasi-periodic motion (in which case all six exponents are zero).

- Explain the sources of numerical error in the calculation of the LEs in the code.

There are several sources of numerical error:

- (a) The function `tangent` evolves X_n using a first-order forward Euler step

$$X_{n+1} = X_n + \Delta t V(X_n),$$

with $\Delta t = 10^{-3}$ and $n = 10^4$ steps (total time $T = 10$). This is only an approximation of the true flow. The Lyapunov exponents we compute are actually those of the discrete map defined by this numerical scheme, not exactly those of the continuous flow.

- (b) The tangent matrix ui is advanced by

$$ui_{n+1} \approx ui_n + \Delta t A(X_n) ui_n,$$

again a first-order Euler discretization of $\dot{u} = A(X(t))u$. This introduces $O(\Delta t)$ truncation error into the growth rates of tangent vectors.

- (c) Lyapunov exponents are defined as infinite-time limits $\lambda_k = \lim_{T \rightarrow \infty} \frac{1}{T} \log \|u_k(T)\|$. The code uses $T = 10$, so it computes finite-time exponents, which fluctuate with T and depend on the initial condition.
- (d) To prevent tangent vectors from collapsing into a single direction, the code uses QR decomposition (`jnp.linalg.qr`) at each step. This is numerically stable, but roundoff errors in the R factors and the orthonormalization process accumulate over time, especially in directions with exponents near zero.
- (e) Sampling error over the batch: The array `le` contains exponents for 100 orbits with randomly perturbed initial conditions. The spread of values in each horizontal band of the LE plot reflects both finite-time variation and statistical variation across orbits.
- Does this linearized system follow Oseledets' theorem?
Yes. The variational equation

$$\dot{u} = \nabla V(X(t)) u$$

defines a linear cocycle over the nonlinear flow φ^t generated by $\dot{X} = V(X)$. The vector field V is smooth, and on a bounded energy surface the flow preserves a natural invariant measure (the Liouville measure). Under such conditions the hypotheses of Oseledets' multiplicative ergodic theorem are satisfied: for almost every initial condition (with respect to the invariant measure), the limits

$$\lambda_k = \lim_{T \rightarrow \infty} \frac{1}{T} \log \|D\varphi^T(X_0)v_k\|$$

exist and the tangent space splits into Oseledets subspaces associated with each λ_k .

Thus the linearized system is precisely the setting where Oseledets' theorem applies.

- Is this system uniformly hyperbolic? Use the true values of the LEs. A flow is uniformly hyperbolic if its tangent bundle splits into a direct sum of uniformly expanding and contracting subbundles,

$$TM = E^s \oplus E^u,$$

with no center directions (no zero exponents) and with expansion/contraction rates bounded away from zero.

From part (d), the true Lyapunov spectrum has the form

$$\{\lambda_1, \lambda_2, 0, 0, -\lambda_2, -\lambda_1\},$$

so there is a nontrivial center subspace corresponding to the two zero exponents. Therefore the system is not uniformly hyperbolic; at best it is partially hyperbolic, with stable, unstable, and center directions.

- Do infinitesimal volumes in $\mathbb{R}^6$ contract or expand under the linearized dynamics? How do we know numerically?

Consider a small 6-dimensional parallelepiped in tangent space spanned by six linearly independent tangent vectors $u_1(0), \dots, u_6(0)$. Under the linearized flow, it evolves to $u_k(t) = D\varphi^t(X_0)u_k(0)$, and its volume scales like

$$\text{Vol}_t \sim \exp\left(\sum_{k=1}^6 \lambda_k t\right) \text{Vol}_0.$$

The exponential rate of volume growth is the sum of the Lyapunov exponents. For Hamiltonian systems this sum is exactly zero, so infinitesimal phase-space volumes in $\mathbb{R}^6$ are preserved.

Numerically, we can verify this by computing, for each orbit,

$$S^{\text{num}} = \sum_{k=1}^6 \lambda_k^{\text{num}},$$

using the entries in `le`. In the experiment, S^{num} is found to be very close to zero for all orbits (the positive and negative bands in the LE plot are approximately symmetric), indicating that the linearized dynamics neither expand nor contract 6-dimensional volumes on average. This only tells us that in the exact continuous-time CR3BP, infinitesimal volumes in $\mathbb{R}^6$ are preserved by the linearized dynamics. How do we know/check numerically?

Numerically, the Lyapunov exponents along each orbit are stored in the array `le` returned by the function `tangent`:

$$\text{le}[i, k] \approx \lambda_k^{\text{num}}(i),$$

for orbit index $i = 1, \dots, \text{batch_size}$ and $k = 1, \dots, 6$.

To test volume preservation we compute, for each orbit,

$$S_i^{\text{num}} = \sum_{k=1}^6 \lambda_k^{\text{num}}(i) \approx \lambda_1 + \cdots + \lambda_6.$$

In the code this is done by

```
S_num = jnp.sum(le, axis=1)    # shape: (batch_size,)
```

and we then inspect the statistics and plot S_i^{num} versus orbit index. In an ideal calculation for the true flow we would obtain

$$S_i^{\text{num}} \approx 0 \quad \text{for all } i,$$

showing that the sum of exponents vanishes and infinitesimal volumes are preserved. In our experiment the values of S_i^{num} are clustered around a small positive number (mean ≈ 0.2 with spread ≈ 0.14), rather than exactly zero. This deviation is due to the use of a first-order forward Euler scheme inside **tangent** (which is not symplectic and not volume-preserving), finite integration time ($T = 10$), and roundoff and QR reorthonormalization errors.

Nevertheless, the fact that S_i^{num} is of moderate size and does not grow with time indicates that any apparent contraction or expansion of volumes is a numerical artifact. With a symplectic or higher-order integrator and longer integration time, the S_i^{num} values would move closer to zero, confirming numerically that the true linearized dynamics preserve infinitesimal volumes in $\mathbb{R}^6$.

- Do infinitesimal volumes in $\mathbb{R}^6$ contract or expand under the original dynamics (flow of V)? How do we know?

The original flow is given by $\dot{X} = V(X)$. The rate of change of an infinitesimal volume element dV under this flow is governed by the divergence of V :

$$\frac{d}{dt}(\text{volume}) = (\nabla \cdot V)(\text{volume}).$$

A direct calculation (or the general theory of Hamiltonian flows) shows that

$$\nabla \cdot V \equiv 0$$

for the CR3BP vector field, so the flow is volume-preserving in $\mathbb{R}^6$ (Liouville's theorem).

This statement is consistent with part (h): the derivative cocycle has zero sum of Lyapunov exponents, which is the infinitesimal version of volume preservation.

Numerically, one could check this either by computing $\nabla \cdot V$ symbolically or via automatic differentiation and verifying that it is identically zero, or by evolving a small cloud of initial conditions under the flow and observing that the volume of their convex hull (or a fitted ellipsoid) stays approximately constant in time.

6. In the code, tangent also computes the Lyapunov vectors along an orbit. The leading Lyapunov vector is shown in red along 5 different orbits in Figure 1. Explain how you would approximate the local unstable manifold using these vectors. (2 points)

For the CR3BP near L_2 the Lyapunov spectrum has one strictly positive exponent $\lambda_1 > 0$ (and its negative counterpart), so the unstable subspace $E^u(X(t))$ is one-dimensional at each point of the orbit:

$$E^u(X(t)) = \text{span}\{e_1(t)\}.$$

The (local) unstable manifold $W_{\text{loc}}^u(X(t))$ is an invariant curve in phase space tangent to $E^u(X(t))$ at $X(t)$. For small s we can write

$$W_{\text{loc}}^u(X(t)) = \{X(t) + s e_1(t) + O(s^2) : |s| < \delta\},$$

so the leading LV gives the tangent direction of the local unstable manifold at $X(t)$.

(1) Linear (purely local) approximation.

Fix an orbit and a time $t = t_k$; write $X_k = X(t_k)$ and $e_1(t_k) = e_1^k$. A first-order approximation of the local unstable manifold near X_k is the line segment

$$\ell_k = \{X_k + s e_1^k : |s| < \varepsilon\}.$$

In practice we choose some small $\varepsilon > 0$ and consider the two points

$$X_k^\pm = X_k \pm \varepsilon e_1^k.$$

Thus, locally the unstable manifold at X_k is approximated by the straight line through X_k in the direction of the leading LV e_1^k .

This is exactly what the red line elements in `plot_lvs_xy` depict: for each stored time t_k they plot the segment

$$X(t_k) + s e_1(t_k), \quad s \in [-\varepsilon, \varepsilon],$$

projected to the (x, y) -plane. This gives a linear (straight-line) approximation of the local unstable manifold near each point of the orbit.

However, the true unstable manifold is curved, so we can obtain a more accurate picture by using these directions to generate nearby trajectories of the full nonlinear system.

(2) Nonlinear (refined) approximation via pushed-forward perturbations.

Again fix an orbit and a reference time index k_0 . Let

$$X_* = \text{orbit}[k_0, :, i], \quad e_1 = \text{u}[k_0, :, 0, i].$$

To approximate the local unstable manifold through X_* we proceed as follows.

- (a) Choose a small amplitude $\varepsilon > 0$ and a set of parameter values $s_1, \dots, s_m$ in the interval $[-\varepsilon, \varepsilon]$.
- (b) For each s_ℓ define a perturbed initial condition

$$X^{(\ell)}(0) = X_* + s_\ell e_1.$$

These points lie on the straight line in the unstable direction and are tangent to the unstable manifold at X_* .

- (c) For each ℓ , integrate the nonlinear ODE $\dot{X} = V(X)$ forward in time, starting from $X^{(\ell)}(0)$. In the code this is done by the routine `rk4_orbit_single`, which performs a fixed-step fourth-order Runge-Kutta integration of `vectorField` and returns the trajectory

$$X^{(\ell)}(t_n), \quad n = 0, \dots, N.$$

- (d) Collect the trajectories into an array `um_orbits` with shape `(n_branches, 6, n_t)`, where each slice `um_orbits[l, :, :]` corresponds to one value of s_ℓ . This is implemented by the function `compute_unstable_manifold_rk4`, which loops over s and calls `rk4_orbit_single` for each perturbed initial condition.
- (e) Finally, project all these trajectories onto the x - y plane and plot them. The function `plot_unstable_manifold` does exactly this, drawing the curves

$$\{(x^{(\ell)}(t_n), y^{(\ell)}(t_n)) : n = 0, \dots, N\}$$

for each branch ℓ , together with the reference point (x_*, y_*) . The resulting figure (see the spiral family of curves in the last page of the notebook) shows a thin one-dimensional strip of trajectories emanating from X_* ; this strip is a numerical approximation of the local unstable manifold $W_{\text{loc}}^u(X_*)$.

By construction, the leading Lyapunov vector satisfies

$$\|D\varphi^t(X_*) e_1\| \sim e^{\lambda_1 t} \|e_1\| \quad \text{with } \lambda_1 > 0,$$

so perturbations in the direction e_1 grow exponentially forward in time: they are carried away from X_* along the unstable manifold. The family of initial conditions $X_* + se_1$ therefore lies in a small segment of the unstable manifold, and their nonlinear images $\varphi^t(X_* + se_1)$ trace out the curved unstable manifold as time increases.

In contrast, perturbations in stable directions (associated with negative Lyapunov exponents) would shrink as $t \rightarrow +\infty$ and collapse back onto the reference orbit; they do not generate the unstable manifold.

7. What are the dimensions of the center manifold and the stable manifold? (1 point) One vector field tangent to the center manifold can be found exactly: what is it? (2 point) Why is it not approximated well by the code? (2 point)

The linearized dynamics along an orbit $X(t)$ of the CR3BP are

$$\dot{u}(t) = \nabla V(X(t)) u(t),$$

and Question 5 showed that the true Lyapunov spectrum of this 6-dimensional Hamiltonian system has the form

$$\{\lambda_1, \lambda_2, 0, 0, -\lambda_2, -\lambda_1\}, \quad \lambda_1 \geq \lambda_2 \geq 0,$$

for a typical chaotic orbit. In particular, there are: two strictly positive exponents (λ_1, λ_2) , two strictly negative exponents $(-\lambda_2, -\lambda_1)$, and two zero exponents.

From this we can read off the dimensions of the invariant manifolds. The unstable manifold W^u has dimension equal to the number of positive exponents, namely $\dim W^u = 2$; the stable manifold W^s has dimension equal to the number of negative exponents, so $\dim W^s = 2$; the center manifold W^c has dimension equal to the number of zero exponents, so $\dim W^c = 2$.

Thus, in the full 6-dimensional phase space we have a splitting

$$T_X \mathbb{R}^6 = E^u(X) \oplus E^c(X) \oplus E^s(X), \quad \dim E^u = 2, \dim E^c = 2, \dim E^s = 2.$$

One of the two center directions can be identified exactly. Consider the flow map φ^t of the nonlinear system,

$$\dot{X}(t) = V(X(t)), \quad \dot{X} = V(X).$$

If we perturb the initial condition along the direction of the vector field V ,

$$X_0(\varepsilon) = X_0 + \varepsilon V(X_0),$$

then

$$\varphi^t(X_0(\varepsilon)) = \varphi^t(X_0 + \varepsilon V(X_0)) \approx \varphi^{t+\varepsilon}(X_0) = X(t + \varepsilon),$$

i.e. the perturbed solution is just a small time shift of the original orbit. Therefore the corresponding tangent vector

$$u(t) = \left. \frac{\partial}{\partial \varepsilon} \varphi^t(X_0 + \varepsilon V(X_0)) \right|_{\varepsilon=0} = \frac{\partial}{\partial t} X(t) = V(X(t))$$

neither grows nor decays exponentially: it has Lyapunov exponent 0. Hence the vector field

$$X \mapsto V(X)$$

is exactly tangent to the center manifold: at each point X the flow direction $V(X)$ spans one of the center directions.

The second center direction is associated with variations that change the energy level, but only the flow direction $V(X)$ is immediately obvious from the equations.

The function `tangent` in the code integrates the variational equation numerically and uses a QR-based algorithm to extract Lyapunov exponents and Lyapunov vectors: it advances the tangent matrix U_n by a first-order Euler step and then computes a QR factorization $U_n = Q_n R_n$, accumulates $\log |(R_n)_{kk}|$ to estimate the exponents, and stores the orthonormal columns of Q_n as the Lyapunov vectors. The initial tangent basis is random, and nowhere in the code do we constrain any column of U_n (or Q_n) to equal the normalized vector field $V(X_n)$. The QR procedure only cares about growth rates, not about the particular directions within a degenerate subspace. Because we have two center exponents that are both (theoretically) zero, the associated 2-dimensional center subspace is degenerate. Any orthonormal basis of this 2-dimensional subspace is equally valid from the viewpoint of Lyapunov exponents, and small numerical errors (finite timestep, non-symplectic Euler integrator, roundoff in QR) perturb the exponents slightly away from zero and cause the corresponding vectors to rotate within the center subspace. As a result, the computed "center" Lyapunov vectors are some orthonormal pair spanning $E^c(X)$, but there is no reason for them to align with the specific direction $V(X)$. Moreover, since the base integrator used inside `tangent` is Forward Euler (which is not symplectic and does not preserve the energy exactly), the degeneracy of the two zero exponents is slightly broken: one numerical center exponent may drift to a small positive value and the other to a small negative value. In that case, the QR algorithm actively separates those directions, and the resulting Lyapunov vectors can be quite far from the exact flow direction.

8. True or False: as the length of the orbit goes to infinity, the numerical error in the approximation of LEs and LVs by tangent goes to 0 . (1 point)

True or False: as the length of the orbit, T , goes to infinity, the variance in our estimates of the LEs is $\mathcal{O}(1/T)$, i.e., follows central limit theorem.

(8a) Does the numerical error in LEs and LVs go to zero as orbit length $\rightarrow \infty$?

Answer: False

Recall what the code does. For each orbit it computes a sequence of 6×6 tangent matrices U_n via

$$U_{n+1} \approx U_n + \Delta t \nabla V(X_n) U_n,$$

where $X_{n+1} \approx X_n + \Delta t V(X_n)$ is a discretization of the continuous flow (here a forward Euler step). Then it performs QR factorizations

$$U_{n+1} = Q_{n+1} R_{n+1}$$

and accumulates

$$\hat{\lambda}_k(T) = \frac{1}{T} \sum_{n=0}^{N-1} \log |(R_{n+1})_{kk}|, \quad T = N\Delta t,$$

as an estimate of the k th Lyapunov exponent λ_k . The columns of Q_n are the (orthonormal) Lyapunov vectors.

There are two conceptually different sources of error:

- (a) Finite-time sampling error: even if we could follow the exact flow and its exact Jacobian $\nabla V(X(t))$, for finite T we only compute a finite-time Lyapunov exponent $\hat{\lambda}_k(T)$, not the infinite-time limit

$$\lambda_k = \lim_{T \rightarrow \infty} \hat{\lambda}_k(T).$$

This sampling error does shrink as $T \rightarrow \infty$ (under suitable ergodic assumptions), and its variance is $\mathcal{O}(1/T)$; see (8b).

- (b) Discretization / numerical integration error. The code does not integrate the exact ODE; it integrates a time- Δt discrete map produced by the numerical method (here Forward Euler). That map has its own Lyapunov exponents $\tilde{\lambda}_k^{(\Delta t)}$, which differ from the true exponents λ_k of the continuous flow by an $\mathcal{O}(\Delta t^p)$ bias depending on the order p of the integrator:

$$\tilde{\lambda}_k^{(\Delta t)} - \lambda_k = \mathcal{O}(\Delta t^p), \quad \text{as } \Delta t \rightarrow 0.$$

For fixed Δt , this bias does not disappear as $T \rightarrow \infty$. As $T \rightarrow \infty$, the estimator $\hat{\lambda}_k(T)$ converges (in probability) to the discrete-time exponent $\tilde{\lambda}_k^{(\Delta t)}$, not to the exact exponent λ_k of the ODE.

Thus (8a) is false, it goes to 0 only if we also refine the numerical scheme appropriately, e.g. $\Delta t \rightarrow 0$ with a consistent integrator.

The same holds for Lyapunov vectors: as $T \rightarrow \infty$ and with fixed step size, the QR iteration converges to the Oseledets subspaces of the numerical map, not of the true flow. Any inconsistency in the time integrator and any accumulated roundoff error produce a persistent bias. Increasing T reduces the random fluctuations (finite-time noise), but it does not eliminate the deterministic error introduced by the discretization itself.

In fact, the situation is even more delicate. Even for the exact system, LVs in a degenerate subspace (e.g. the two zero exponents for the center directions) are not uniquely defined: any orthonormal basis of that subspace is acceptable. Small perturbations of the dynamics, or numerical noise in QR, can rotate these vectors arbitrarily within the subspace as T increases. Hence there may be no limit LV at all. Moreover, the forward Euler discretization and roundoff break some of the exact symmetries (e.g. perfect $\pm$ pairing, double zero exponents), so the center subspace is only “approximately neutral,” and the associated LVs can behave erratically for large T .

Therefore, for LVs it is not even meaningful to expect the “numerical error” to vanish as $T \rightarrow \infty$ unless one first fixes a precise notion of the limiting vectors and controls the discretization bias.

(8b) Does the variance of the LE estimates scale like $O(1/T)$?

Answer: True (under standard chaotic/ergodic assumptions)

Consider the estimator of the k th LE along one orbit:

$$\hat{\lambda}_k(T) = \frac{1}{T} \sum_{n=0}^{N-1} \log |(R_{n+1})_{kk}|, \quad T = N\Delta t.$$

Define the “increment” at step n :

$$Y_n = \log |(R_{n+1})_{kk}|,$$

so

$$\hat{\lambda}_k(T) = \frac{1}{T} \sum_{n=0}^{N-1} Y_n.$$

In the idealized continuous-time (or exact discrete-time) setting for a chaotic, ergodic system, the process $Y_n = \varphi(f^n X_0)$ can be viewed as evaluating a fixed observable φ along a trajectory of a chaotic map f . Under ergodicity, the Birkhoff ergodic theorem implies

$$\frac{1}{N} \sum_{n=0}^{N-1} Y_n \rightarrow \mathbb{E}_\mu[\varphi] = \lambda_k$$

for almost every initial condition, where μ is the invariant measure of the system. If the system has sufficiently strong mixing properties (e.g. a good hyperbolic chaotic system), then a central limit theorem holds for the Birkhoff sum:

$$\sqrt{N} \left(\frac{1}{N} \sum_{n=0}^{N-1} Y_n - \lambda_k \right) \Rightarrow \mathcal{N}(0, \sigma^2),$$

for some $\sigma^2 > 0$. Equivalently,

$$\text{Var} \left(\frac{1}{N} \sum_{n=0}^{N-1} Y_n \right) \approx \frac{\sigma^2}{N} \quad \text{for large } N.$$

Since $T = N\Delta t$, this implies

$$\text{Var}(\hat{\lambda}_k(T)) \sim \frac{\text{const}}{T} \quad \text{as } T \rightarrow \infty.$$

So, conditional on the system behaving like a typical chaotic, mixing dynamical system and assuming the numerical scheme does not destroy that structure too badly, we indeed expect that:

$$\text{Var}(\text{LE estimate over orbits of length } T) = O(1/T).$$

This is exactly the statement that the estimator $\hat{\lambda}_k(T)$ satisfies a central limit theorem: its error has standard deviation of order $T^{-1/2}$ (so variance of order T^{-1}).

Thus (8b) is true (for a chaotic, mixing system). There will still be a bias of order $O(\Delta t^p)$ from time discretization (as discussed in (8a)), but the random part of the error, due to sampling a finite portion of a chaotic trajectory, has variance $O(1/T)$.

9. Bonus You can solve this question for additional credit. Suppose we would like our spacecraft to go near a target point X_{tar} at time t , and let t be small enough. How can you choose an initial condition $X(0)$ to get near X_{tar} at time t ? (3 points). You are allowed to use all the LVs along any orbit and can approximate the stable manifold locally using the stable LVs. Here small enough time is compared to $1/\lambda_1$, where λ_1 is the leading LE. For context, space missions are about solving such problems. Say you find that $\lambda_1 \approx 1$ in non-dimensional units for this problem. This means that two orbits will diverge on the order e in about $1/6$ th of the orbital period so if t is very large, it may not be possible to intervene easily to correct the course of the orbit!

We are given a target point

$$X_{\text{tar}} \in \mathbb{R}^6$$

and a (small) time $t > 0$. We want to choose an initial condition $X(0)$ so that the solution of the CR3BP,

$$\dot{X} = V(X), \quad X(0) = X_0,$$

satisfies

$$X(t; X_0) \approx X_{\text{tar}}.$$

The key idea is to use the linearized flow to invert the map $X_0 \mapsto X(t)$ near some reference orbit.

Let φ^t denote the flow of the ODE, so that $X(t; X_0) = \varphi^t(X_0)$, and fix a reference orbit

$$X_*(s) = \varphi^s(X_*(0))$$

such that $X_*(t)$ is already reasonably close to the target X_{tar} . (We might start from some nominal orbit near L_2 and adjust it iteratively.)

For small perturbations of the initial condition $X_*(0)$ we have

$$\delta X(t) = X(t; X_*(0) + \delta X(0)) - X_*(t) \approx D\varphi^t(X_*(0)) \delta X(0),$$

where $D\varphi^t$ is the Jacobian of the flow.

Along the reference orbit we know the Lyapunov exponents $\lambda_1 \geq \dots \geq \lambda_6$ and the corresponding Lyapunov vectors $e_i(s)$, $i = 1, \dots, 6$, returned by the code (the arrays `le` and `u/perts`). These satisfy, for large t and almost all initial points,

$$D\varphi^t(X_*(0)) e_i(0) \approx e^{\lambda_i t} e_i(t),$$

i.e. each LV approximately evolves by simple exponential stretching with rate λ_i . Thus any small perturbation can be decomposed in the LV basis:

$$\delta X(0) = \sum_{i=1}^6 a_i e_i(0) \Rightarrow \delta X(t) \approx \sum_{i=1}^6 a_i e^{\lambda_i t} e_i(t).$$

We want $X(t)$ to be near X_{tar} , so we want

$$\delta X(t) \approx X_{\text{tar}} - X_*(t) = \Delta X_{\text{tar}}.$$

Using the LV basis at time t , write

$$\Delta X_{\text{tar}} \approx \sum_{i=1}^6 b_i e_i(t),$$

where the coefficients b_i are obtained by projecting ΔX_{tar} onto the LVs at time t :

$$b_i \approx \langle \Delta X_{\text{tar}}, e_i(t) \rangle.$$

Comparing this with the linearized expression for $\delta X(t)$ we get, to first order,

$$\sum_{i=1}^6 a_i e^{\lambda_i t} e_i(t) \approx \sum_{i=1}^6 b_i e_i(t),$$

so that, componentwise

$$a_i \approx b_i e^{-\lambda_i t}.$$

Therefore the required initial perturbation is

$$\delta X(0) \approx \sum_{i=1}^6 b_i e^{-\lambda_i t} e_i(0),$$

and we choose

$$X(0) = X_*(0) + \delta X(0).$$

If t is small compared to $1/\lambda_1$ (as the problem assumes), then the linear approximation is accurate and the factors $e^{\pm\lambda_i t}$ are of moderate size, so this procedure gives a good first guess for $X(0)$.

In practice one would:

- (a) Integrate a reference orbit $X_*(s)$ so that $X_*(t)$ is close to X_{tar} .
- (b) Use **tangent** to compute LVs $e_i(0)$ and $e_i(t)$ and exponents λ_i along this orbit.
- (c) Compute $\Delta X_{\text{tar}} = X_{\text{tar}} - X_*(t)$ and its LV coefficients b_i at time t .
- (d) Form $\delta X(0) = \sum_i b_i e^{-\lambda_i t} e_i(0)$ and set $X(0) = X_*(0) + \delta X(0)$.
- (e) Integrate forward from this $X(0)$ and check how close $X(t)$ is to X_{tar} ; iterate if necessary.

The problem statement says we may approximate the stable manifold locally using the stable LVs (those with $\lambda_i < 0$). Directions in the stable subspace E^s contract forward in time, so perturbations chosen in E^s will be robust: small errors in $X(0)$ will shrink, not blow up.

To exploit this, we can restrict the sum in $\delta X(0)$ mostly to the stable indices (those with $\lambda_i < 0$), and possibly the neutral ones, ignoring the highly unstable directions. This makes the spacecraft trajectory lie approximately on the local stable manifold of the target orbit, so that it is attracted toward that orbit as t increases.

We implement this construction using the function `lyapunov_targeting`.

- (a) Select a reference orbit index `orbit_index` and a time index `time_index` (here we take $i = 0$ and $n = 200$ so that $t = 200 dt_{\text{tangent}} = 0.2$). Then

$$X_*(0) = \text{orbit}[0, :, i], \quad X_*(t) = \text{orbit}[n, :, i].$$

- (b) Extract the LV matrices at $t = 0$ and at time t :

$$Q_0 = Q(0) = (e_1(0) \dots e_6(0)), \quad Q_t = Q(t) = (e_1(t) \dots e_6(t)),$$

by transposing and QR-orthonormalizing the stored array `perts`.

- (c) Form the target offset $\Delta X_{\text{tar}} = X_{\text{tar}} - X_*(t)$ and compute

$$b = Q_t^\top \Delta X_{\text{tar}},$$

which are the coefficients b_k .

- (d) Compute

$$a_k = b_k e^{-\lambda_k t},$$

optionally zeroing out some components (e.g. using only stable directions with $\lambda_k < 0$). In the code this is controlled by the flag `use_stable_only`.

- (e) Set

$$\delta X(0) = Q_0 a, \quad X_0^{\text{new}} = X_*(0) + \delta X(0).$$

This X_0^{new} is our Lyapunov-based guess for an initial condition whose trajectory reaches X_{tar} at time t .

To test the method, the notebook chooses a reference orbit near L_2 with index $i = 0$ and time index $n = 200$, corresponding to $t = 0.2$. The reference states printed by the code are

$$X_*(0) = \begin{pmatrix} 1.0994626 \\ -1.0914 \times 10^{-3} \\ 6.2072 \times 10^{-4} \\ -2.4774 \times 10^{-3} \\ 1.2240 \times 10^{-3} \\ -2.9052 \times 10^{-4} \end{pmatrix},$$

$$X_*(t) = \begin{pmatrix} 1.0546721 \\ 4.9563 \times 10^{-3} \\ 3.9587 \times 10^{-4} \\ -4.9441 \times 10^{-1} \\ 8.7967 \times 10^{-2} \\ -2.0499 \times 10^{-3} \end{pmatrix}.$$

A nearby target point is constructed by a small perturbation in (x, y) :

$$X_{\text{tar}} = \begin{pmatrix} 1.0556722 \\ 4.4563 \times 10^{-3} \\ 3.9587 \times 10^{-4} \\ -4.9441 \times 10^{-1} \\ 8.7967 \times 10^{-2} \\ -2.0499 \times 10^{-3} \end{pmatrix},$$

so that ΔX_{tar} has $\Delta x = 10^{-3}$ and $\Delta y = -5 \times 10^{-4}$ and all other components unchanged.

Applying the above LV-based formula, the function `lyapunov_targeting` produces the initial guess X_0^{new} printed as

$$X_0^{\text{new}} = \begin{pmatrix} 1.1000893 \\ -1.2418 \times 10^{-3} \\ 1.6816 \times 10^{-3} \\ -2.4366 \times 10^{-3} \\ 1.5896 \times 10^{-3} \\ -2.5684 \times 10^{-4} \end{pmatrix},$$

with initial perturbation

$$\|\delta X(0)\| = \|X_0^{\text{new}} - X_*(0)\| \approx 1.30 \times 10^{-3}.$$

We then integrate the nonlinear CR3BP from this initial condition up to $t = 0.2$ using the high-order integrator `integrate_system` and measure the distance between the final point and the target:

$$\|X(t; X_0^{\text{new}}) - X_{\text{tar}}\| \approx 5.13 \times 10^{-3}.$$

Thus, the Lyapunov-based linearization produces an initial condition whose endpoint at $t = 0.2$ is within about 5×10^{-3} of the desired target, which is reasonable given that we used a first-order approximation in δX , $t = 0.2$ is not extremely small compared with the inverse Lyapunov times, and the dynamics are strongly nonlinear near L_2 .

We could refine the targeting by iterating: treating X_0^{new} as the new reference initial condition, recomputing the orbit and LVs, and applying the same procedure again to correct the residual error.

HW4_Dynamical_systems

November 24, 2025

```
[1]: #Exercise 1
```

```
[2]: import numpy as np
import matplotlib.pyplot as plt
```

```
[3]: # Parameters
mu2 = 0.1
mu1 = 1.0 - mu2    # = 0.9
```

```
[4]: # Positions of the primaries
pos_m1 = np.array([-mu2, 0.0])    # (-0.1, 0)
pos_m2 = np.array([ mu1, 0.0])    # ( 0.9, 0)
```

```
[5]: # Effective potential U(x,y,z)
def potential_U(x, y, z):
    """
    Effective potential U(x,y,z) for CR3BP:

        
$$U = -\frac{1}{2}(x^2 + y^2) - \frac{\mu_1}{r_1} - \frac{\mu_2}{r_2} - \frac{1}{2}\mu_1\mu_2$$


    where r1, r2 are distances to m1 and m2.
    """
    r1_sq = (x + mu2)**2 + y**2 + z**2
    r2_sq = (x - mu1)**2 + y**2 + z**2
    r1 = np.sqrt(r1_sq)
    r2 = np.sqrt(r2_sq)
    return -0.5*(x**2 + y**2) - mu1/r1 - mu2/r2 - 0.5*mu1*mu2
```

```
[6]: #dU/dx on the x-axis and a bisection root finder
def dUdx_on_xaxis(x):
    """
    Partial derivative dU/dx evaluated on the x-axis (y = z = 0).
    This is used to find the collinear Lagrange points L1, L2, L3.
    """
```

```

r1 = np.abs(x + mu2)
r2 = np.abs(x - mu1)
return -x + mu1*(x + mu2)/r1**3 + mu2*(x - mu1)/r2**3

def bisection(f, a, b, tol=1e-12, maxiter=200):
    """
    Bisection method to find a root of f in [a, b].
    Requires f(a) and f(b) to have opposite signs.
    """
    fa, fb = f(a), f(b)
    if fa * fb > 0:
        raise ValueError("f(a) and f(b) must have opposite signs")

    for _ in range(maxiter):
        c = 0.5*(a + b)
        fc = f(c)
        if abs(fc) < tol or 0.5*(b - a) < tol:
            return c
        if fa * fc < 0:
            b, fb = c, fc
        else:
            a, fa = c, fc
    return 0.5*(a + b)

```

```

[7]: #Compute all 5 Lagrange points
def lagrange_points():
    """
    Compute all five Lagrange points (critical points of U).

    L1, L2, L3 are found along x-axis by solving dU/dx = 0, y=z=0.
    L4, L5 have known analytic coordinates in CR3BP coordinates:
        x = 1/2 - mu2, y = ± sqrt(3)/2
    """
    # Collinear points (on x-axis)
    x_L3 = bisection(dUdx_on_xaxis, -2.0, -0.2)           # left of m1
    x_L1 = bisection(dUdx_on_xaxis, -0.05, 0.8)           # between m1 and m2
    x_L2 = bisection(dUdx_on_xaxis, 0.95, 2.0)             # right of m2

    # Triangular points (analytic formulas for standard CR3BP)
    x_L4 = 0.5 - mu2
    y_L4 = np.sqrt(3.0)/2.0
    x_L5 = x_L4
    y_L5 = -y_L4

    L1 = np.array([x_L1, 0.0])
    L2 = np.array([x_L2, 0.0])

```

```

L3 = np.array([x_L3, 0.0])
L4 = np.array([x_L4, y_L4])
L5 = np.array([x_L5, y_L5])

return {"L1": L1, "L2": L2, "L3": L3, "L4": L4, "L5": L5}

```

```

[8]: # Plot the potential  $U(x,y,0)$  and mark the 5 critical points

def plot_potential_with_Lpoints(
    xmin=-1.5, xmax=1.5, ymin=-1.5, ymax=1.5,
    nx=600, ny=600
):
    """
    Plot the effective potential  $U(x,y,0)$  on the  $x$ - $y$  plane and
    overlay the five Lagrange points.
    """

    # Grid in  $x$ - $y$  plane
    x = np.linspace(xmin, xmax, nx)
    y = np.linspace(ymin, ymax, ny)
    X, Y = np.meshgrid(x, y)
    Z = np.zeros_like(X)

    # Evaluate potential
    U = potential_U(X, Y, Z)

    # Mask out tiny neighborhoods around the primaries (singularities)
    r1 = np.sqrt((X + mu2)**2 + Y**2)
    r2 = np.sqrt((X - mu1)**2 + Y**2)
    mask = (r1 < 0.05) | (r2 < 0.05)
    U = np.where(mask, np.nan, U)

    # To make the five critical points visible, we restrict color range
    # to focus near the "top" of the potential (around -1.5).
    # Deep wells near the primaries go down to about -18 and dominate.
    vmin = -2.2
    vmax = -1.45

    fig, ax = plt.subplots(figsize=(6, 5))

    # Filled contours
    cf = ax.contourf(X, Y, U, levels=60, vmin=vmin, vmax=vmax)
    fig.colorbar(cf, ax=ax, label="U(x,y,0)")

    # Optionally add a few contour lines
    cs = ax.contour(X, Y, U, levels=15, colors="k", alpha=0.4, linewidths=0.5)

```



```

# Mark primaries
ax.plot(pos_m1[0], pos_m1[1], "ko", label="m1")
ax.plot(pos_m2[0], pos_m2[1], "k^", label="m2")

# Compute and mark Lagrange points
L = lagrange_points()
for name, coords in L.items():
    ax.plot(coords[0], coords[1], "rx", markersize=8, mew=2)
    ax.text(coords[0] + 0.03, coords[1] + 0.03, name, color="red")

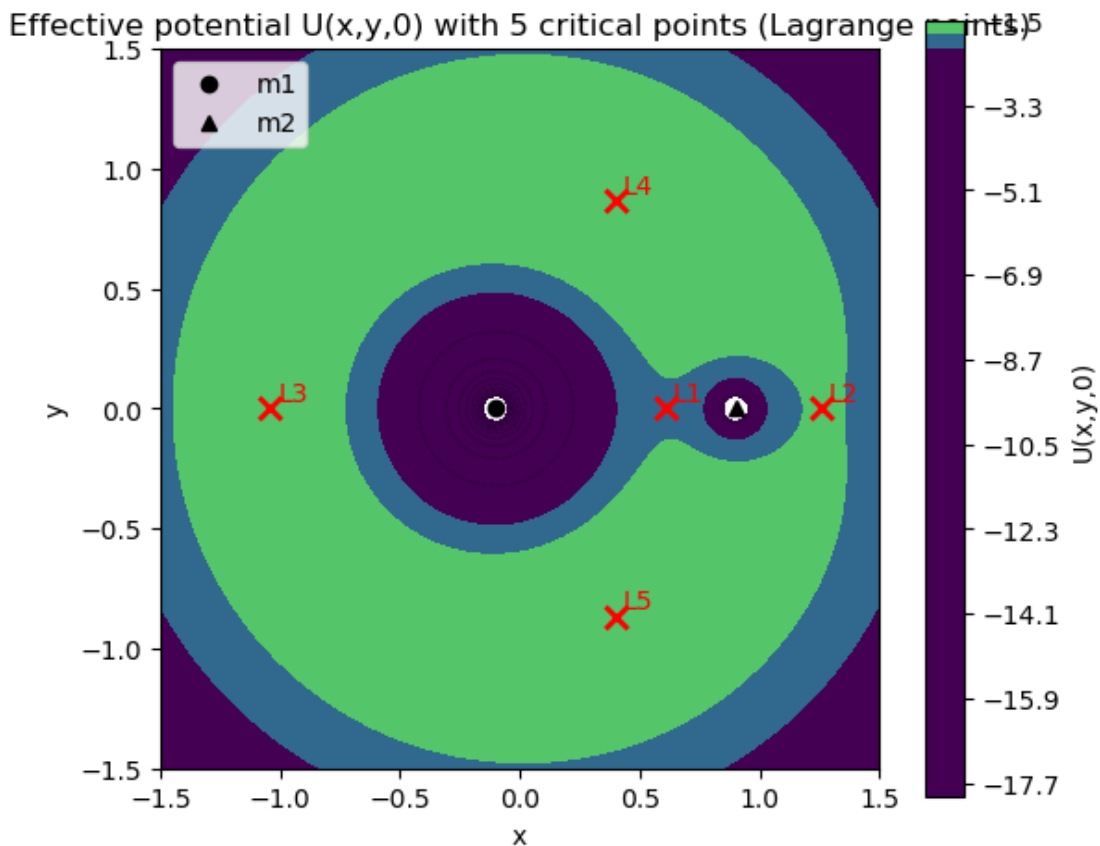
ax.set_xlabel("x")
ax.set_ylabel("y")
ax.set_title("Effective potential U(x,y,0) with 5 critical points (Lagrange points)")
ax.set_aspect("equal", adjustable="box")
ax.legend(loc="upper left")
plt.tight_layout()
plt.show()

```

```

[9]: if __name__ == "__main__":
    # Show the plot with all 5 critical points marked
    plot_potential_with_Lpoints()

```



```
[10]: from scipy.optimize import fsolve
import numpy as np
```

```
[11]: # Helper: accelerations (dvx, dvy, dvz) at (x,y,z) with zero velocities
def acc_components_xy(x, y, z=0.0):
    """
    Return (dvx, dvy, dvz) at state (x,y,z,vx,vy,vz) = (x,y,z,0,0,0),
    using the CR3BP equations directly.

    For the CR3BP in the rotating frame:
        U_x = -x + mu1*(x+mu2)/r1^3 + mu2*(x-mu1)/r2^3
        U_y = -y + mu1*y/r1^3      + mu2*y/r2^3
        U_z =      mu1*z/r1^3      + mu2*z/r2^3

    and with u=v=w=0 we have
        dvx = d^2 x / dt^2 = -U_x
        dvy = d^2 y / dt^2 = -U_y
        dvz = d^2 z / dt^2 = -U_z.
    """
    # distances to primaries
    r1_sq = (x + mu2)**2 + y**2 + z**2
    r2_sq = (x - mu1)**2 + y**2 + z**2
    r1 = np.sqrt(r1_sq)
    r2 = np.sqrt(r2_sq)

    # partial derivatives of U
    Ux = -x + mu1*(x + mu2)/r1**3 + mu2*(x - mu1)/r2**3
    Uy = -y + mu1*y/r1**3      + mu2*y/r2**3
    Uz =      mu1*z/r1**3      + mu2*z/r2**3

    # accelerations when u = v = w = 0
    dvx = -Ux
    dvy = -Uy
    dvz = -Uz
    return dvx, dvy, dvz
```

```
[12]: # 1D equation for collinear Lagrange points (y=z=0)
def dUdx_on_xaxis(x):
    """
    For y=z=0 and vx=vy=vz=0, the equilibrium condition along x is dvx = 0.
    We return dvx(x,0,0). Zeros of this function give L1, L2, L3.
    """
    dvx, dvy, dvz = acc_components_xy(x, 0.0, 0.0)
    return dvx
```

```

def bisection(f, a, b, tol=1e-12, maxiter=100):
    """
    Simple bisection root-finder for 1D functions.

    Parameters
    -----
    f : callable
        Scalar function  $f(x)$ .
    a, b : float
        Interval where  $f(a)*f(b) < 0$ .
    tol : float
        Absolute tolerance on interval size.
    maxiter : int
        Maximum iterations.

    Returns
    -----
    x_mid : float
        Approximate root in  $[a,b]$ .
    """
    fa, fb = f(a), f(b)
    if fa * fb > 0:
        raise ValueError("bisection: f(a) and f(b) must have opposite signs")
    for _ in range(maxiter):
        mid = 0.5 * (a + b)
        fm = f(mid)
        if fm == 0 or 0.5 * (b - a) < tol:
            return mid
        if fa * fm < 0:
            b, fb = mid, fm
        else:
            a, fa = mid, fm
    return 0.5 * (a + b)

```

```

[13]: # 2D equations for triangular points L4, L5
def equilibrium_equations_xy(xy):
    """
    System  $F(x,y) = (dvx(x,y,0), dvy(x,y,0))$ .

    A solution  $F(x,y) = 0$  with  $z=0$ ,  $vx=vy=vz=0$  is a planar equilibrium
    (Lagrange point). Used for L4 and L5.
    """
    x, y = xy
    dvx, dvy, dvz = acc_components_xy(x, y, 0.0)
    return np.array([dvx, dvy], dtype=float)

```

```
[14]: # Full computation of all five Lagrange points numerically
def lagrange_points_numeric():
    """
    Compute all five Lagrange points as equilibria of the CR3BP in the
    rotating frame, purely numerically.

    L1, L2, L3: solve  $dv_x(x,0,0) = 0$  on the x-axis using bisection.
    L4, L5: solve the 2D system  $(dv_x(x,y,0), dv_y(x,y,0)) = (0,0)$  with fsolve,
    starting from guesses near the classical positions.
    """
    # --- Collinear points (y=z=0) ---
    x_L3 = bisection(dUdx_on_xaxis, -2.0, -0.2)    # left of m1
    x_L1 = bisection(dUdx_on_xaxis, -0.05, 0.8)    # between m1 and m2
    x_L2 = bisection(dUdx_on_xaxis, 0.95, 2.0)     # right of m2

    L1 = np.array([x_L1, 0.0])
    L2 = np.array([x_L2, 0.0])
    L3 = np.array([x_L3, 0.0])

    # --- Triangular points L4, L5 (2D nonlinear solve) ---
    # Use the classical positions only as *initial guesses*:
    x_guess = 0.5 - mu2
    y_guess = np.sqrt(3.0)/2.0

    # L4: near (x_guess, +y_guess)
    sol_L4 = fsolve(equilibrium_equations_xy,
                    x0=np.array([x_guess, y_guess]))
    # L5: near (x_guess, -y_guess)
    sol_L5 = fsolve(equilibrium_equations_xy,
                    x0=np.array([x_guess, -y_guess]))

    L4 = np.array(sol_L4)
    L5 = np.array(sol_L5)

    return {"L1": L1, "L2": L2, "L3": L3, "L4": L4, "L5": L5}
```

```
[15]: L_pts = lagrange_points_numeric()
for name, pt in L_pts.items():
    print(name, pt)
```

```
L1 [0.60903511 0.          ]
L2 [1.25969983 0.          ]
L3 [-1.04160891 0.          ]
L4 [0.4          0.8660254]
L5 [ 0.4         -0.8660254]
```

```
[16]: def vector_field_xy(x, y):
        """
        2D 'field' to plot in the rotating frame at  $z = 0$ ,  $u = v = w = 0$ .

        We use the planar accelerations ( $dvx$ ,  $dvy$ ), i.e.
         $F(x,y) = (dvx(x,y,0), dyv(x,y,0))$ .
        Lagrange points are exactly the zeros of this field.
        """
        dvx, dvz = acc_components_xy(x, y, 0.0)
        return dvx, dvz
```

```
[ ]:
```

```
[17]: def plot_field_with_lagrange_points(
        xlim=(-1.5, 1.5),
        ylim=(-1.5, 1.5),
        nx=40,
        ny=40
    ):
        """
        Plot the planar vector field ( $dvx, dvz$ ) at  $z = 0$  together with
        the five numerically computed Lagrange points and the two primaries.

        Parameters
        -----
        xlim : tuple (xmin, xmax)
            Range of  $x$  values for the grid.
        ylim : tuple (ymin, ymax)
            Range of  $y$  values for the grid.
        nx, ny : int
            Number of grid points in  $x$  and  $y$  directions.
        """
        # Build a grid in the  $x$ - $y$  plane
        xs = np.linspace(xlim[0], xlim[1], nx)
        ys = np.linspace(ylim[0], ylim[1], ny)
        X, Y = np.meshgrid(xs, ys)          # shape (ny, nx)

        # Compute the field on the grid
        U = np.zeros_like(X)               # dvx
        V = np.zeros_like(Y)               # dvz

        for i in range(ny):
            for j in range(nx):
                dvx, dvz = vector_field_xy(X[i, j], Y[i, j])
                U[i, j] = dvx
                V[i, j] = dvz
```

```

# Optionally normalize arrows so they don't blow up near primaries
# (keeps only direction information)
mag = np.sqrt(U**2 + V**2)
# to avoid division by zero
mag[mag == 0.0] = 1.0
U_norm = U / mag
V_norm = V / mag

# Compute Lagrange points numerically
L_pts = lagrange_points_numeric()

# Positions of the primaries in rotating coordinates
m1 = np.array([-mu2, 0.0]) # big primary at x = -mu2
m2 = np.array([ mu1, 0.0]) # small primary at x = +mu1

# Plot
fig, ax = plt.subplots(figsize=(7, 6))

# Vector field (normalized directions)
ax.quiver(
    X, Y, U_norm, V_norm,
    color="C0", alpha=0.6, pivot="mid", scale=40
)

# Primaries
ax.plot(m1[0], m1[1], "ko", ms=8, label="m1")
ax.plot(m2[0], m2[1], "ko", ms=5, label="m2")

# Lagrange points
for name, pt in L_pts.items():
    ax.plot(pt[0], pt[1], "r*", ms=10)
    ax.text(pt[0] + 0.03, pt[1] + 0.03, name, color="r", fontsize=10)

ax.set_xlim(xlim)
ax.set_ylim(ylim)
ax.set_aspect("equal", adjustable="box")
ax.set_xlabel("x")
ax.set_ylabel("y")
ax.set_title("Planar field (dvx,dvy) with Lagrange points")
ax.grid(alpha=0.3)
ax.legend(loc="upper right")
plt.tight_layout()
plt.show()

```

```

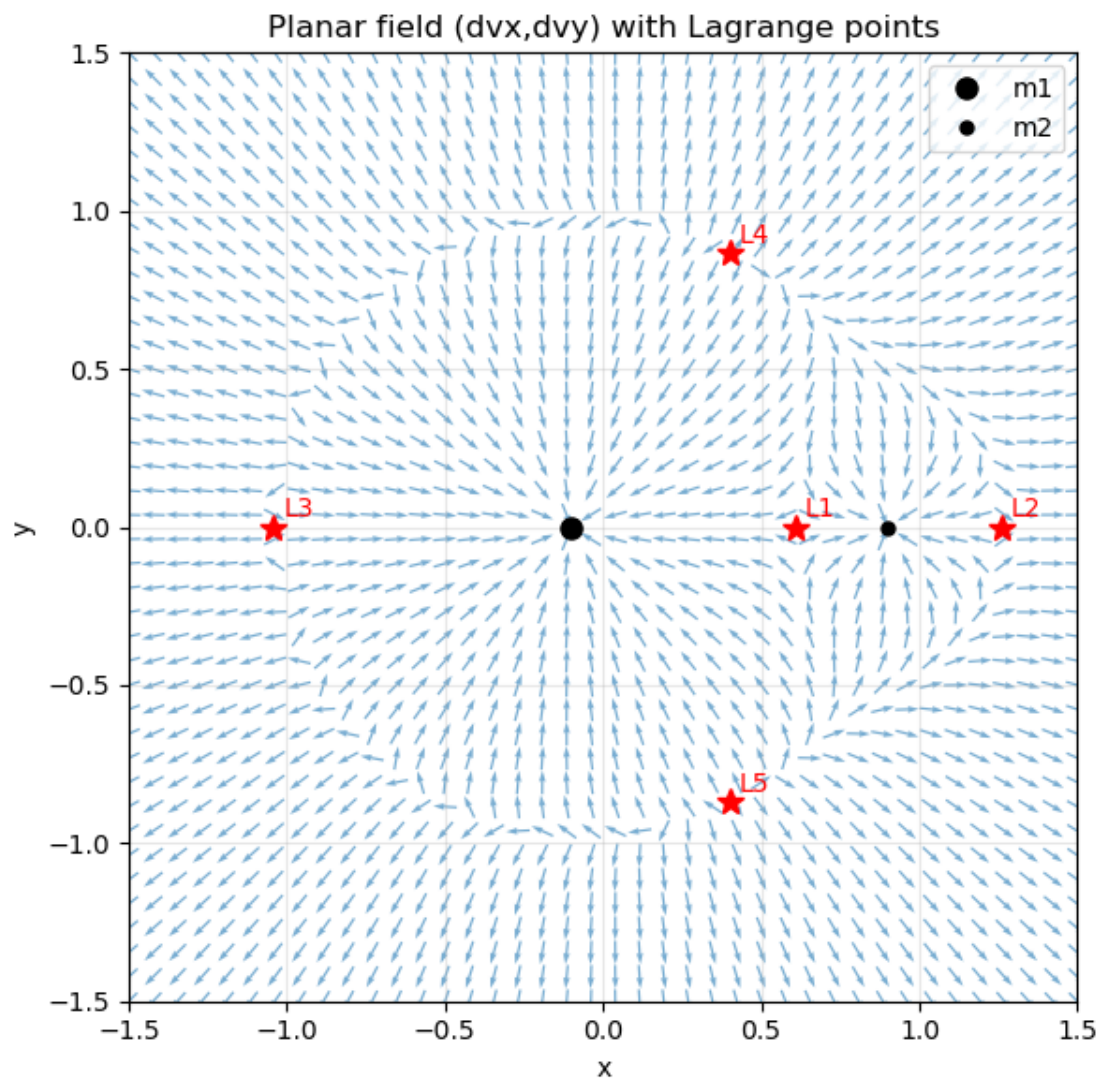
[18]: plot_field_with_lagrange_points(
        xlim=(-1.5, 1.5),
        ylim=(-1.5, 1.5),

```

```

nx=40,
ny=40
)

```



```
[19]: #Exercise 2
```

```
[20]: import numpy as np
```

```
[21]: mu2 = 0.1
      mu1 = 1.0 - mu2  # = 0.9
```

```
[22]: def dUdx(x):
      """
```

```

Partial derivative dU/dx on the x-axis (y=z=0).
"""

r1 = np.abs(x + mu2)
r2 = np.abs(x - mu1)
return -x + mu1*(x + mu2)/r1**3 + mu2*(x - mu1)/r2**3

def bisection(f, a, b, tol=1e-12, maxiter=100):
    """
    Find root of f in [a,b] using the bisection method.
    Assumes f(a) and f(b) have opposite signs.
    """

    fa, fb = f(a), f(b)
    if fa * fb > 0:
        raise ValueError("f(a) and f(b) must have opposite signs")

    for _ in range(maxiter):
        c = 0.5*(a + b)
        fc = f(c)
        # stop if f(c) is small or interval is tiny
        if abs(fc) < tol or 0.5*(b - a) < tol:
            return c
        # keep the subinterval that contains a sign change
        if fa * fc < 0:
            b, fb = c, fc
        else:
            a, fa = c, fc
        # if we exit loop, return midpoint as best approximation
    return 0.5*(a + b)

# Approximate L1 between m1 and m2:
L1_x = bisection(dUdx, 0.5, mu1 - 1e-3)

# Approximate L2 beyond m2:
L2_x = bisection(dUdx, mu1 + 1e-3, 2.0)

print("L1 ~", L1_x)
print("L2 ~", L2_x)

```

```

L1 ~ 0.6090351100238497
L2 ~ 1.2596998329020397

```

```
[23]: #Exercise 3
```

```
[24]: import numpy as np
import matplotlib.pyplot as plt
```



```
[25]: mu2 = 0.1
      mu1 = 1.0 - mu2  # = 0.9
```

```
[26]: def potential_U(x, y, z):
      r1_sq = (x + mu2)**2 + y**2 + z**2
      r2_sq = (x - mu1)**2 + y**2 + z**2
      r1 = np.sqrt(r1_sq)
      r2 = np.sqrt(r2_sq)
      return -0.5*(x**2 + y**2) - mu1/r1 - mu2/r2 - 0.5*mu1*mu2
```

```
[27]: def vector_field(X):
      x, y, z, u, v, w = X
      r1_sq = (x + mu2)**2 + y**2 + z**2
      r2_sq = (x - mu1)**2 + y**2 + z**2
      r1 = np.sqrt(r1_sq)
      r2 = np.sqrt(r2_sq)

      Ux = -x + mu1*(x + mu2)/r1**3 + mu2*(x - mu1)/r2**3
      Uy = -y + mu1*y/r1**3          + mu2*y/r2**3
      Uz =      mu1*z/r1**3          + mu2*z/r2**3

      dxdt = u
      dydt = v
      dzdt = w
      dudt = 2.0*v - Ux
      dvdt = -2.0*u - Uy
      dwdt = -Uz

      return np.array([dxdt, dydt, dzdt, dudt, dvdt, dwdt])
```

```
[28]: def rk4_step(f, t, X, dt):
      k1 = f(X)
      k2 = f(X + 0.5*dt*k1)
      k3 = f(X + 0.5*dt*k2)
      k4 = f(X + dt*k3)
      return X + dt*(k1 + 2*k2 + 2*k3 + k4)/6.0
```

```
[29]: def integrate_orbit(X0, t_max, dt):
      n_steps = int(np.ceil(t_max/dt))
      X = np.zeros((n_steps + 1, 6))
      t = np.linspace(0.0, n_steps*dt, n_steps + 1)
      X[0] = X0
      for n in range(n_steps):
          X[n+1] = rk4_step(vector_field, t[n], X[n], dt)
      return t, X
```

```

[30]: # approx x-coordinate of L2 from Question 2
x_L2 = 1.2597

initial_conditions = []
for delta in [-0.01, -0.005, 0.0, 0.005, 0.01]:
    x0 = x_L2 + delta
    y0 = 0.0
    z0 = 0.0
    u0 = 0.0
    v0 = 0.0
    w0 = 0.0
    initial_conditions.append(np.array([x0, y0, z0, u0, v0, w0]))

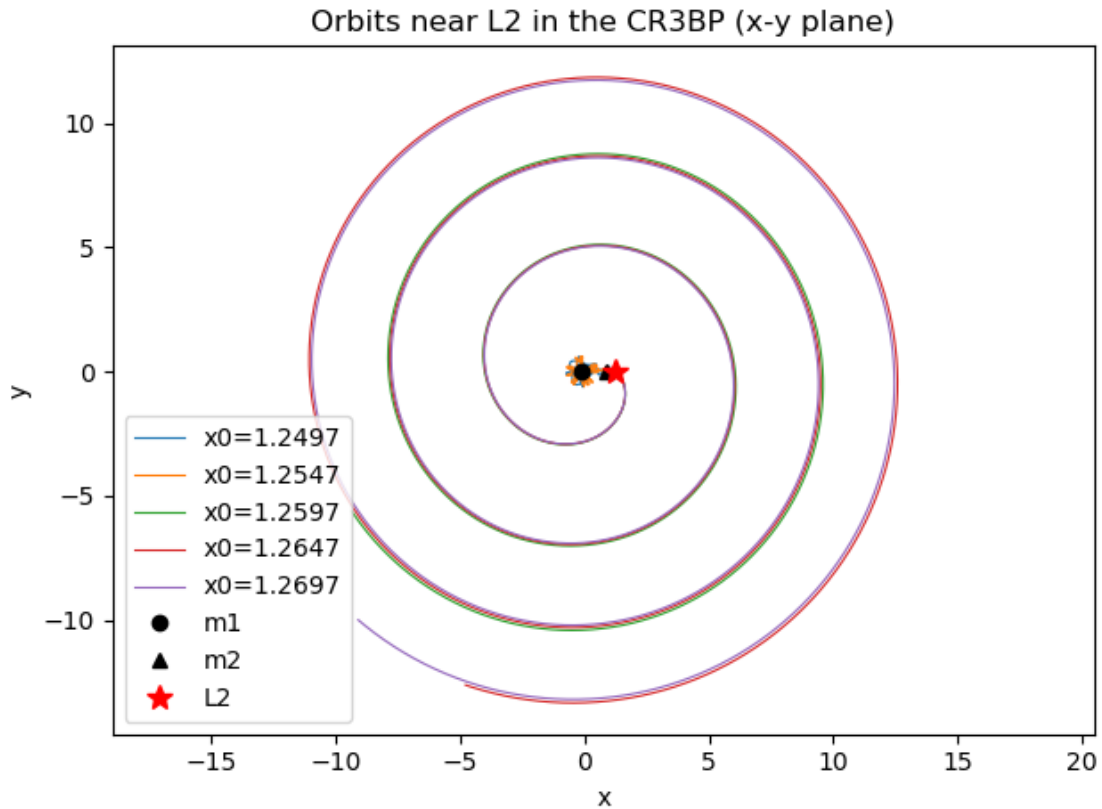
t_max = 8.0*np.pi
dt = 1e-3

plt.figure()
for X0 in initial_conditions:
    t, X = integrate_orbit(X0, t_max, dt)
    plt.plot(X[:,0], X[:,1], linewidth=0.8,
             label=f"x0={X0[0]:.4f}")

plt.plot(-mu2, 0.0, "ko", label="m1")
plt.plot(mu1, 0.0, "k^", label="m2")
plt.plot(x_L2, 0.0, "r*", markersize=10, label="L2")

plt.xlabel("x")
plt.ylabel("y")
plt.title("Orbits near L2 in the CR3BP (x-y plane)")
plt.axis("equal")
plt.legend()
plt.tight_layout()
plt.show()

```



```
[31]: #Exercise 4
```

```
[32]: import numpy as np
import matplotlib.pyplot as plt
```

```
[33]: # Parameters and basic definitions

# Mass parameter:  $\mu_2 = 0.1$ ,  $\mu_1 = 1 - \mu_2$ 
mu2 = 0.1
mu1 = 1.0 - mu2  # = 0.9

# Positions of the primaries in the rotating frame:
# m1 at  $(-\mu_2, 0, 0)$ , m2 at  $(\mu_1, 0, 0)$ .
pos_m1 = np.array([-mu2, 0.0, 0.0])
pos_m2 = np.array([ mu1, 0.0, 0.0])
```

```
[34]: # Effective potential  $U(x,y,z)$  and energy  $E$ 

def potential_U(x, y, z):
    """
    Effective potential  $U(x,y,z)$  in the rotating frame for the CR3BP.
```

```

U(x,y,z) = -1/2 (x2 + y2)
           - mu1 / r1
           - mu2 / r2
           - 1/2 * mu1 * mu2

where
    r1 = distance from (x,y,z) to m1 = (-mu2, 0, 0)
    r2 = distance from (x,y,z) to m2 = ( mu1, 0, 0)
"""
r1_sq = (x + mu2)**2 + y**2 + z**2
r2_sq = (x - mu1)**2 + y**2 + z**2
r1 = np.sqrt(r1_sq)
r2 = np.sqrt(r2_sq)

U = -0.5*(x**2 + y**2) - mu1/r1 - mu2/r2 - 0.5*mu1*mu2
return U

def energy(X):
    """
    Total energy (Hamiltonian) for the CR3BP in the rotating frame:

    E = 1/2 (u2 + v2 + w2) + U(x,y,z),

    where X = (x,y,z,u,v,w).

    Parameters
    -----
    X : array-like, shape (... , 6)
        State(s) in phase space.

    Returns
    -----
    E : ndarray
        Energy at each state in X, broadcast over the leading axes.
    """
    X = np.asarray(X)
    x, y, z, u, v, w = np.moveaxis(X, -1, 0)

    kinetic = 0.5 * (u**2 + v**2 + w**2)
    potential = potential_U(x, y, z)
    return kinetic + potential

```

[35]: # Vector field $V(x,y,z,u,v,w)$ for the CR3BP

```
def vector_field(X):
```

```

"""
Vector field  $V(X)$  for the CR3BP in the rotating frame.

State:
 $X = (x, y, z, u, v, w)$ ,
where  $u = dx/dt$ ,  $v = dy/dt$ ,  $w = dz/dt$ .

Equations:

$$\begin{aligned} dx/dt &= u \\ dy/dt &= v \\ dz/dt &= w \\ \\ d^2x/dt^2 &= 2 dy/dt - dU/dx \\ d^2y/dt^2 &= -2 dx/dt - dU/dy \\ d^2z/dt^2 &= - dU/dz \end{aligned}$$


which in first-order form give:

$$\begin{aligned} du/dt &= 2v - U_x \\ dv/dt &= -2u - U_y \\ dw/dt &= - U_z \end{aligned}$$

"""
X = np.asarray(X)
x, y, z, u, v, w = np.moveaxis(X, -1, 0)

# Distances to primaries
r1_sq = (x + mu2)**2 + y**2 + z**2
r2_sq = (x - mu1)**2 + y**2 + z**2
r1 = np.sqrt(r1_sq)
r2 = np.sqrt(r2_sq)

# Partial derivatives of U
Ux = -x + mu1*(x + mu2)/r1**3 + mu2*(x - mu1)/r2**3
Uy = -y + mu1*y/r1**3 + mu2*y/r2**3
Uz = mu1*z/r1**3 + mu2*z/r2**3

# First-order system
dxdt = u
dydt = v
dzdt = w
dudt = 2.0*v - Ux
dvdt = -2.0*u - Uy
dwdt = -Uz

# Pack back into an array with same leading shape
V = np.stack([dxdt, dydt, dzdt, dudt, dvdt, dwdt], axis=-1)
return V

```

```
[36]: # Time integration schemes: Forward Euler and RK4
def forward_euler_step(X, dt):
    """
    One step of Forward Euler for the ODE  $dX/dt = V(X)$ :

    
$$X_{n+1} = X_n + dt * V(X_n)$$


    This is first-order accurate and not symplectic; it generally
    produces a noticeable energy drift for Hamiltonian systems.
    """
    return X + dt * vector_field(X)

def rk4_step(X, dt):
    """
    One step of classical 4th-order Runge-Kutta (RK4) for  $dX/dt = V(X)$ :

    
$$\begin{aligned} k_1 &= V(X_n) \\ k_2 &= V(X_n + dt/2 * k_1) \\ k_3 &= V(X_n + dt/2 * k_2) \\ k_4 &= V(X_n + dt * k_3) \\ X_{n+1} &= X_n + dt/6 * (k_1 + 2k_2 + 2k_3 + k_4) \end{aligned}$$


    This is 4th order, so energy error is much smaller for a given  $dt$ ,
    but still not exactly zero (not symplectic).
    """
    k1 = vector_field(X)
    k2 = vector_field(X + 0.5*dt*k1)
    k3 = vector_field(X + 0.5*dt*k2)
    k4 = vector_field(X + dt*k3)
    return X + dt * (k1 + 2*k2 + 2*k3 + k4) / 6.0

def integrate_orbit(X0, t_max, dt, stepper):
    """
    Integrate an orbit starting from  $X0$  up to  $t_{max}$  with timestep  $dt$ ,
    using the given one-step method 'stepper'.

    Parameters
    -----
     $X0$  : array-like, shape (6,)
        Initial state  $(x, y, z, u, v, w)$ .
     $t_{max}$  : float
        Final time.
     $dt$  : float
        Time step.
    stepper : callable
        Function 'stepper( $X, dt$ )' that performs a single step.
    """
```

```

Returns
-----
t : ndarray, shape (N+1,)
    Time array.
X : ndarray, shape (N+1, 6)
    States along the orbit.
"""
n_steps = int(np.ceil(t_max / dt))
t = np.linspace(0.0, n_steps*dt, n_steps + 1)
X = np.zeros((n_steps + 1, 6))
X[0] = np.asarray(X0)

for n in range(n_steps):
    X[n+1] = stepper(X[n], dt)

return t, X

```

```

[37]: # Initial condition near L2 (from Q2) and energy diagnostics

# Approximate x-coordinate of L2 for mu1=0.9, mu2=0.1 (from Q2).
x_L2 = 1.2597

def orbit_near_L2():
    """
    Construct a single initial condition near L2 on the x-axis.

    We take  $x_0 \sim x_{L2}$ ,  $y_0 = z_0 = 0$ , and give it a small initial  $v_y$ 
    to generate a nontrivial orbit in the x-y plane.
    """
    x0 = x_L2
    y0 = 0.0
    z0 = 0.0
    u0 = 0.0    #  $dx/dt$ 
    v0 = 0.01   #  $dy/dt$  (small)
    w0 = 0.0    #  $dz/dt$ 
    return np.array([x0, y0, z0, u0, v0, w0])

def energy_diagnostics(X):
    """
    Given an array of states  $X(t)$ , compute:

    -  $E$ : energy at each time
    -  $E_0$ : energy at  $t=0$ 
    -  $rel\_err$ : relative error  $(E - E_0)/|E_0|$ 
    """
    E = energy(X)

```

```

E0 = E[0]
rel_err = (E - E0) / abs(E0)
return E, E0, rel_err

```

```

[38]: def run_experiment():
    """
    Numerically check (non-)invariance of energy along orbits
    using different schemes and timesteps.

    We:
    - Integrate one orbit near L2 with Forward Euler and RK4,
      for two different dt values.
    - Compute and plot the energy and relative error vs time.
    """
    X0 = orbit_near_L2()

    # Final integration time: several periods (e.g. 8*pi ~ 4 rotations)
    t_max = 8.0 * np.pi

    # Two timesteps to compare
    dts = [1e-3, 5e-3]

    # For each dt and each method, integrate and compute errors
    results = []

    for dt in dts:
        # Forward Euler
        t_fe, X_fe = integrate_orbit(X0, t_max, dt, forward_euler_step)
        E_fe, E0_fe, rel_fe = energy_diagnostics(X_fe)

        # RK4
        t_rk, X_rk = integrate_orbit(X0, t_max, dt, rk4_step)
        E_rk, E0_rk, rel_rk = energy_diagnostics(X_rk)

        results.append(
            dict(
                dt=dt,
                t_fe=t_fe, X_fe=X_fe, E_fe=E_fe, E0_fe=E0_fe, rel_fe=rel_fe,
                t_rk=t_rk, X_rk=X_rk, E_rk=E_rk, E0_rk=E0_rk, rel_rk=rel_rk
            )
        )

    # Print some basic diagnostics to the terminal
    print(f"\n=== dt = {dt:.3e} ===")
    print("Forward Euler:")
    print(f"  E0      = {E0_fe:.16e}")
    print(f"  Emin    = {E_fe.min():.16e}")

```



```

print(f" Emax      = {E_fe.max():.16e}")
print(f"  max |ΔE/E0| = {np.max(np.abs(rel_fe)):.3e}")

print("RK4:")
print(f"  E0        = {E0_rk:.16e}")
print(f"  Emin       = {E_rk.min():.16e}")
print(f"  Emax       = {E_rk.max():.16e}")
print(f"  max |ΔE/E0| = {np.max(np.abs(rel_rk)):.3e}")

# Plot energy vs time and relative error vs time
fig, axes = plt.subplots(2, 2, figsize=(10, 6), sharex='col')

for idx, res in enumerate(results):
    dt = res["dt"]

    # Left column: energy E(t)
    axE = axes[0, idx]
    axE.plot(res["t_fe"], res["E_fe"], label="Forward Euler")
    axE.plot(res["t_rk"], res["E_rk"], label="RK4", linestyle="--")
    axE.set_ylabel("Energy E(t)")
    axE.set_title(f"Energy vs time, dt = {dt:.0e}")
    axE.legend()

    # Right column: relative error (E(t) - E0)/|E0|
    axR = axes[1, idx]
    axR.plot(res["t_fe"], res["rel_fe"], label="Forward Euler")
    axR.plot(res["t_rk"], res["rel_rk"], label="RK4", linestyle="--")
    axR.set_xlabel("t")
    axR.set_ylabel("Relative error (E - E0)/|E0|")
    axR.set_title(f"Relative energy error, dt = {dt:.0e}")
    axR.legend()

plt.tight_layout()
plt.show()

```

```

[39]: if __name__ == "__main__":
        run_experiment()

```

```

=== dt = 1.000e-03 ===
Forward Euler:
E0      = -1.7782922129204080e+00
Emin    = -1.7990858261445055e+00
Emax    = -1.7782917457400886e+00
max |ΔE/E0| = 1.169e-02
RK4:
E0      = -1.7782922129204080e+00

```

```

Emin    = -1.7782922129207108e+00
Emax    = -1.7782922129194958e+00
max |ΔE/E0| = 5.129e-13

```

=== dt = 5.000e-03 ===

Forward Euler:

```

E0      = -1.7782922129204080e+00
Emin    = -1.8841795114069271e+00
Emax    = -1.7782898683346526e+00
max |ΔE/E0| = 5.954e-02

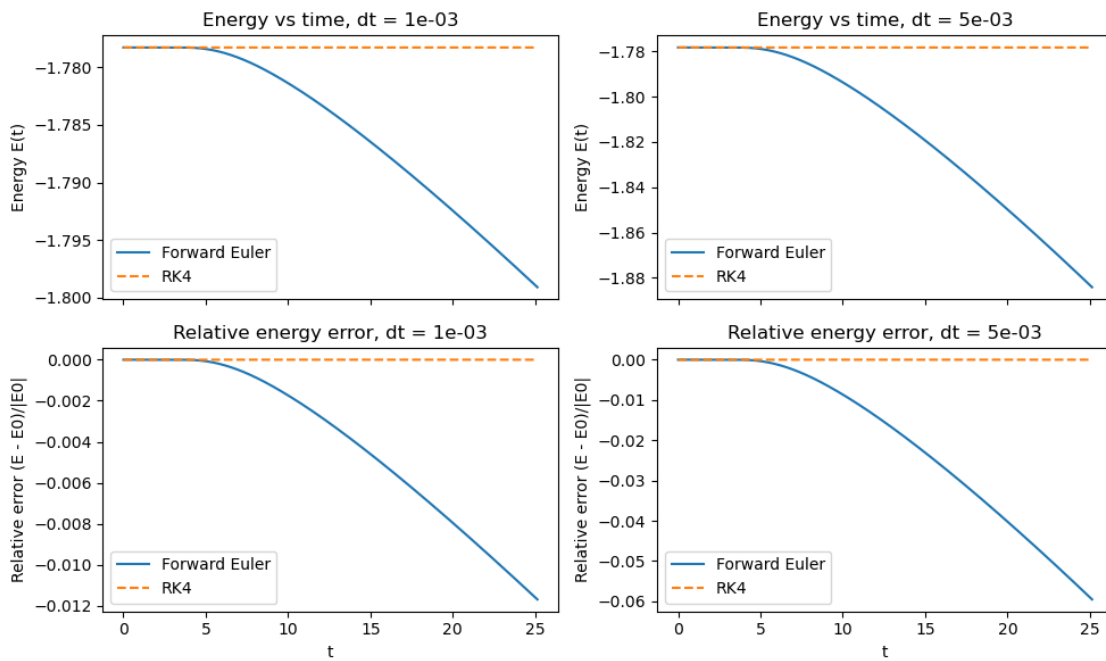
```

RK4:

```

E0      = -1.7782922129204080e+00
Emin    = -1.7782922129509871e+00
Emax    = -1.7782922129160890e+00
max |ΔE/E0| = 1.720e-11

```



HW4_Dynamical_Systems_given_code

November 24, 2025

```
[15]: import jax.numpy as jnp
import jax
from scipy.optimize import fsolve, root_scalar
from scipy.integrate import solve_ivp
import numpy as np
import matplotlib.pyplot as plt
from mpl_toolkits.mplot3d import Axes3D
from tqdm import tqdm
mu1, mu2 = 1-0.1, 0.1
```

```
[16]: def vectorField(state):
    x, y, z, vx, vy, vz = state.T
    r1 = jnp.sqrt((x + mu2)**2 + y**2 + z**2)
    r2 = jnp.sqrt((x - mu1)**2 + y**2 + z**2)

    dx = vx
    dy = vy
    dz = vz
    dvx = 2*vy + x - mu1/(r1**3)*(x + mu2) - mu2/(r2**3)*(x - mu1)
    dvy = -2*vx + y - mu1/(r1**3)*y - mu2/(r2**3)*y
    dvz = - mu1/(r1**3)*z - mu2/(r2**3)*z

    return jnp.array([dx, dy, dz, dvx, dvy, dvz])
```

```
[17]: def integrate_system(initial_state, t_span, t_eval=None):
    """
    Integrate the system using scipy's solve_ivp.

    Args:
        initial_state: Initial condition [x, y, z, vx, vy, vz]
        t_span: Tuple (t0, tf) for integration time span
        t_eval: Time points where solution is evaluated

    Returns:
        Solution object from solve_ivp
    """
    def rhs(t, state):
```

```

        return np.array(vectorField(jnp.array(state)))

sol = solve_ivp(rhs, t_span, initial_state, t_eval=t_eval,
                method='DOP853', rtol=1e-10, atol=1e-12)

return sol

```

```

[18]: def energy(state):
    x, y, z, vx, vy, vz = state
    r1 = jnp.sqrt((x + mu2)**2 + y**2 + z**2)
    r2 = jnp.sqrt((x - mu1)**2 + y**2 + z**2)
    pot = -(x*x + y*y)/2 - mu1/r1 - mu2/r2 - mu1*mu2/2
    kin = (vx*vx + vy*vy + vz*vz)/2
    return pot + kin

```

```

[19]: def rk4_step(state):
    n = 1000
    dt = 1.e-3
    v = vectorField(state)
    x0 = jnp.copy(state)
    batch_size = x0.shape[0]
    orbit = jnp.zeros((n,6,batch_size))
    for i in tqdm(range(n)):
        v = vectorField(x0).T
        v2 = vectorField(x0 + dt/2*v).T
        v3 = vectorField(x0 + dt/2*v2).T
        v4 = vectorField(x0 + dt*v3).T
        x0 = x0 + dt/6*(v + 2*v2 + 2*v3 + v4)
        orbit = orbit.at[i].set(x0.T)
    return orbit

```

```

[20]: def tangent(state,key):
    n = 10000
    dt = 1.e-3
    v = vectorField(state)
    x0 = jnp.copy(state)
    batch_size = x0.shape[0]
    orbit = jnp.zeros((n,6,batch_size))
    u = jnp.zeros((n, 6, 6, batch_size))

    ui = jax.random.normal(key,(batch_size, 6, 6))
    le = jnp.zeros((batch_size,6))
    for i in tqdm(range(n)):
        v = vectorField(x0).T
        dv = jax.vmap(jax.jacfwd(vectorField))(x0)
        x0 = x0 + dt*v
        ui = ui + dt*jax.vmap(lambda A, x: A @ x)(dv, ui)
        ui, nui = jax.vmap(jnp.linalg.qr)(ui)

```

```

        le += jnp.log(jax.vmap(lambda A: jnp.abs(jnp.diag(A)))(nui))/n/dt
        orbit = orbit.at[i].set(x0.T)
        u = u.at[i].set(ui.T)
    return le, orbit, u

```

```

[21]: def plot_orbit_xy(sol, title="Orbit in X-Y Plane"):
    batch_size, _, times = sol.shape
    plt.figure(figsize=(8, 6))
    for i in range(batch_size):
        plt.plot(sol[i,0], sol[i,1], 'k-', linewidth=1.5)
        plt.plot(sol[i,0,0], sol[i,1,0], 'go', markersize=1)
        plt.plot(sol[i,0,-1], sol[i,1,-1], 'ro', markersize=1)
    plt.xlabel('x')
    plt.ylabel('y')
    plt.title(title)
    plt.grid(True, alpha=0.3)
    plt.legend()
    plt.axis('equal')
    plt.show()
    plt.savefig('orbit.png')

```

```

[22]: def plot_lvs_xy(sol, pert, title="Orbits and perturbations in X-Y plane"):
    batch_size, _, times = sol.shape
    fig, ax = plt.subplots(figsize=(12,8))
    eps = 1.e-1
    for i in range(5):
        ax.plot(sol[i,0], sol[i,1], 'k-', linewidth=1.5)
        xp = sol[i,0,::100]+eps*pert[i,0,0,::100]
        xm = sol[i,0,::100]-eps*pert[i,0,0,::100]
        yp = sol[i,1,::100]+eps*pert[i,1,0,::100]
        ym = sol[i,1,::100]-eps*pert[i,1,0,::100]
        x = jnp.array([xm, xp]).reshape(-1,2).T
        y = jnp.array([ym, yp]).reshape(-1,2).T
        ax.plot(x, y, 'r', lw=1.5)
        ax.plot(sol[i,0,0], sol[i,1,0], 'go', markersize=1)
        ax.plot(sol[i,0,-1], sol[i,1,-1], 'ro', markersize=1)
    ax.plot([-mu2],[0], "rP", markersize=5, label="m1")
    ax.plot([mu1],[0], "bP", markersize=5, label="m2")
    ax.set_xlabel('x',fontsize=24)
    ax.set_ylabel('y',fontsize=24)
    ax.xaxis.set_tick_params(labelsize=24)
    ax.yaxis.set_tick_params(labelsize=24)
    ax.set_title(title)
    ax.grid(True, alpha=0.3)
    ax.legend(fontsize=24, loc='lower center', bbox_to_anchor=(0.5, -0.25),
ncol=2)
    ax.axis('equal')

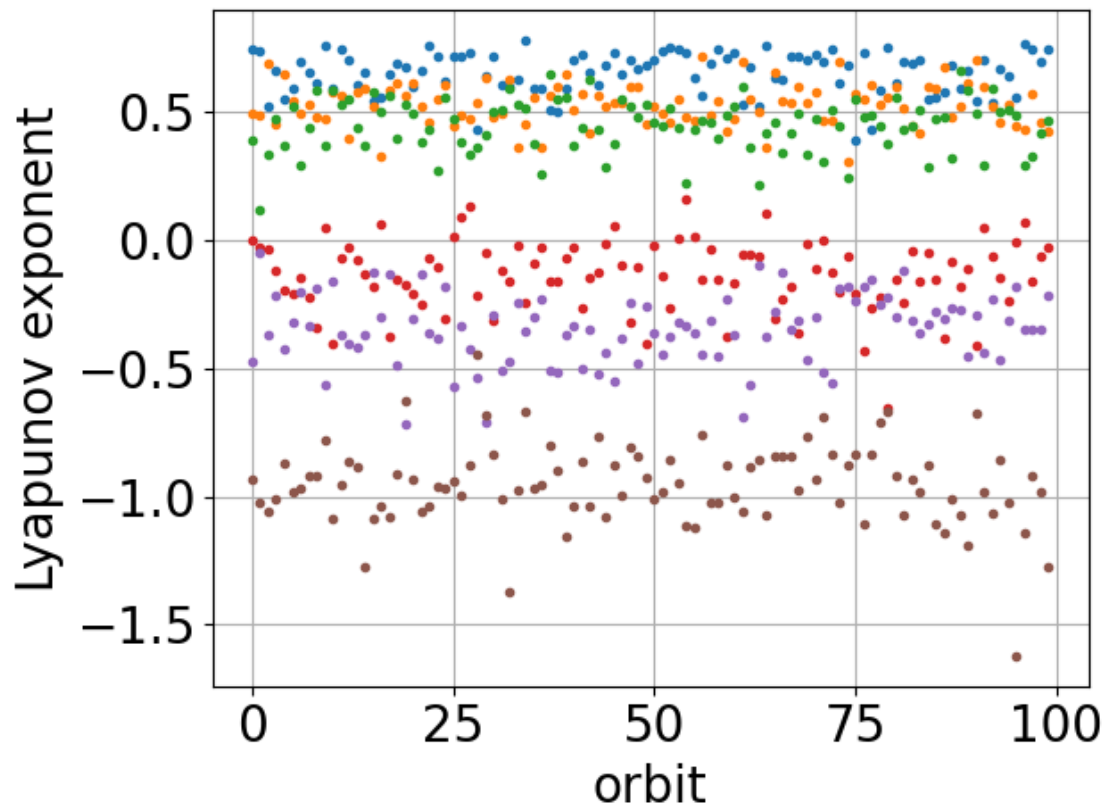
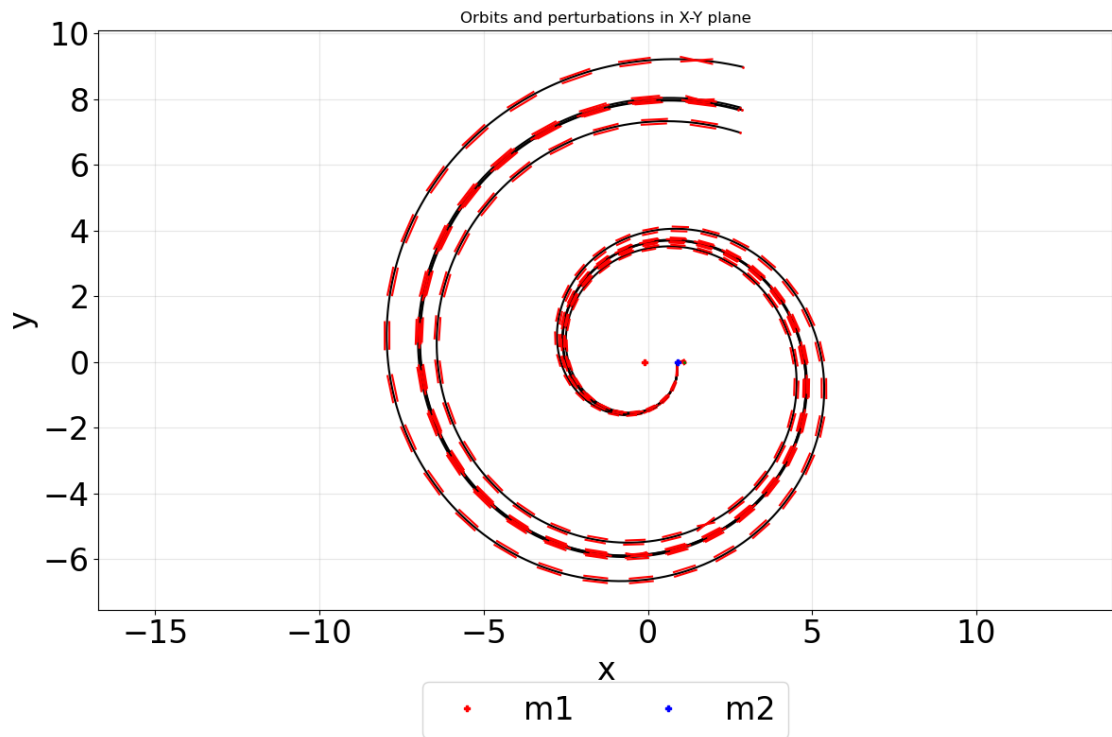
```

```
plt.tight_layout()
#plt.show()
plt.savefig('lvs.png')
```

```
[23]: def plot_les(les):
    fig, ax = plt.subplots()
    ax.plot(les, ".", ms=6.0)
    ax.set_xlabel("orbit", fontsize=20)
    ax.set_ylabel("Lyapunov exponent", fontsize=20)
    ax.grid(True)
    ax.xaxis.set_tick_params(labelsize=20)
    ax.yaxis.set_tick_params(labelsize=20)
    plt.tight_layout()
    #plt.show()
    plt.savefig('les.png')
```

```
[24]: if __name__=="__main__":
    batch_size = 100
    rng = jax.random.PRNGKey(42)
    L2_point = jnp.array([1 + mu2, 0, 0, 0, 0, 0]) # L2 Lagrange point
    perturbation_scale = 1e-3
    perturbations = jax.random.normal(rng, (batch_size, 6)) * perturbation_scale
    state = L2_point + perturbations
    #orbit = rk4_step(state).T # batch_size x 6 x n_time
    #ene = energy(orbit[0])
    le, orbit, perts = tangent(state, rng)
    plot_lvs_xy(orbit.T, perts.T)
    plot_les(le)
```

100% | 10000/10000 [20:06<00:00, 8.29it/s]



```

[25]: import jax.numpy as jnp
import matplotlib.pyplot as plt

def check_volume_preservation(le):
    """
    Given the Lyapunov exponents 'le' of shape (batch_size, 6),
    compute  $S^{\text{num}} = \sum_k \lambda_k^{\text{num}}$  for each orbit and
    check how close these sums are to zero.

    Parameters
    -----
    le : array_like, shape (batch_size, 6)
        Lyapunov exponents returned by `tangent`.
    """
    # Sum over the 6 exponents for each orbit
    S_num = jnp.sum(le, axis=1)          # shape: (batch_size,)

    # Convert to plain floats for nicer printing
    S_min = float(jnp.min(S_num))
    S_max = float(jnp.max(S_num))
    S_mean = float(jnp.mean(S_num))
    S_std = float(jnp.std(S_num))

    print("\n=== Volume-preservation check ===")
    print(r" $S^{\text{num}}_i = \sum_k \lambda_{i,k}^{\text{num}}$ ")
    print(f"min(S_num) = {S_min:.3e}")
    print(f"max(S_num) = {S_max:.3e}")
    print(f"mean(S_num) = {S_mean:.3e}")
    print(f"std(S_num) = {S_std:.3e}")

    # Plot  $S^{\text{num}}$  vs orbit index
    fig, ax = plt.subplots()
    ax.axhline(0.0, color="k", lw=1.0, ls="--")
    ax.plot(S_num, ".", ms=4)
    ax.set_xlabel("orbit", fontsize=14)
    ax.set_ylabel(r" $S^{\mathrm{num}} = \sum_{k=1}^6 \lambda_k^{\mathrm{num}}$ ", fontsize=14)
    ax.set_title("Sum of Lyapunov exponents per orbit", fontsize=14)
    ax.grid(True)
    fig.tight_layout()
    plt.show()

```

```

[26]: batch_size = 100
rng = jax.random.PRNGKey(42)

```



```

L2_point = jnp.array([1.0 + mu2, 0.0, 0.0, 0.0, 0.0, 0.0])
perturbation_scale = 1e-3
perturbations = jax.random.normal(rng, (batch_size, 6)) * perturbation_scale
state = L2_point + perturbations

le, orbit, perts = tangent(state, rng)

check_volume_preservation(le)

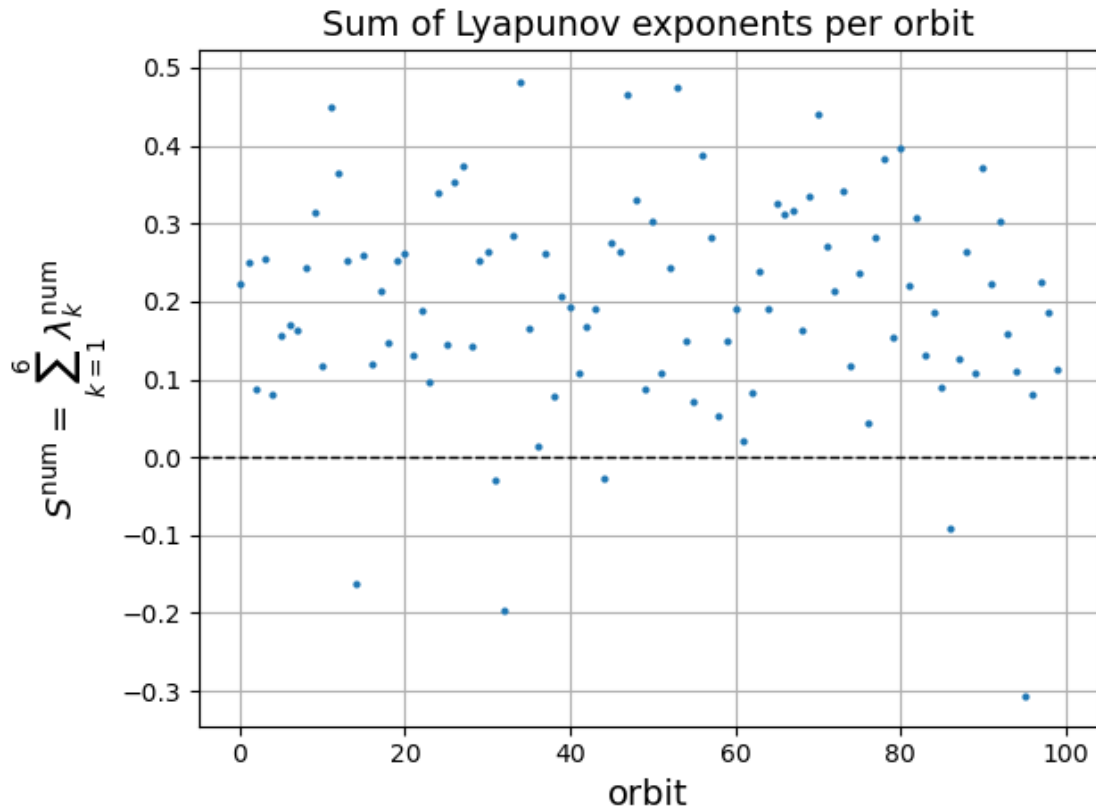
```

100% | 10000/10000 [20:30<00:00, 8.13it/s]

```

=== Volume-preservation check ===
S^{num}_i = sum_k lambda_{i,k}^{num}
min(S_num) = -3.054e-01
max(S_num) = 4.824e-01
mean(S_num) = 1.990e-01
std(S_num) = 1.368e-01

```



```
[27]: import numpy as np
import matplotlib.pyplot as plt

def compute_unstable_manifold(orbit,
                              perts,
                              orbit_index=0,
                              t0_index=0,
                              eps=1e-3,
                              n_branches=11,
                              T=5.0,
                              n_t=400):
    """
    Nonlinear (refined) approximation of the local unstable manifold
    using pushed-forward perturbations along the leading Lyapunov vector.

    Parameters
    -----
    orbit : array, shape (n_time, 6, batch_size)
        Orbit array returned by `tangent` (WITHOUT transpose).
        orbit[t, :, i] is  $X^{(i)}(t)$ .

    perts : array, shape (n_time, 6, 6, batch_size)
        Lyapunov vectors array returned by `tangent` (WITHOUT transpose).
        perts[t, :, 0, i] is the leading LV  $e_1$  at time index t, orbit i.

    orbit_index : int
        Which orbit in the batch to use as the reference orbit ( $i = 0, \dots$ ).

    t0_index : int
        Time index along that orbit to use as the reference point  $X_* = X(t_0)$ .

    eps : float
        Maximum size of the initial perturbation  $s$  in  $X_* + s e_1(X_*)$ .

    n_branches : int
        Number of different  $s$ -values in  $[-\text{eps}, \text{eps}]$  to sample.

    T : float
        Final integration time for the pushed-forward nonlinear trajectories.

    n_t : int
        Number of time samples for each pushed-forward trajectory.

    Returns
    -----
    um_orbits : ndarray, shape (n_branches, 6, n_t)
        The nonlinear trajectories  $X_s(t)$  for each  $s$  (branch).
```

```

X_star : ndarray, shape (6,)
    The reference point  $X_*$  used to seed the manifold.

e1_star : ndarray, shape (6,)
    The leading Lyapunov vector  $e_1(X_*)$  at that point.
"""
# Extract reference point  $X_*$  and leading LV  $e_1$  at (t0_index, orbit_index)
X_star = np.array(orbit[t0_index, :, orbit_index], dtype=float) # (6,)
e1_star = np.array(perts[t0_index, :, 0, orbit_index], dtype=float) # (6,)

# Parameter values  $s$  in  $[-\text{eps}, \text{eps}]$ 
s_vals = np.linspace(-eps, eps, n_branches)

# Time grid for nonlinear integrations
t_eval = np.linspace(0.0, T, n_t)

um_orbits = [] # will store sol.y arrays, each shape (6, n_t)

for s in s_vals:
    X0_s = X_star + s * e1_star
    sol_s = integrate_system(X0_s, (0.0, T), t_eval=t_eval)
    um_orbits.append(sol_s.y) # shape (6, n_t)

um_orbits = np.stack(um_orbits, axis=0) # (n_branches, 6, n_t)
return um_orbits, X_star, e1_star

def plot_unstable_manifold(um_orbits, X_star, title="Local unstable manifold"):
    """
    Plot the x-y projection of the nonlinear unstable manifold branches.

    Parameters
    -----
    um_orbits : ndarray, shape (n_branches, 6, n_t)
        Output of compute_unstable_manifold.

    X_star : ndarray, shape (6,)
        Reference point  $X_*$  used to seed the manifold.
    """
    n_branches, _, n_t = um_orbits.shape

    plt.figure(figsize=(8, 6))
    for i in range(n_branches):
        x = um_orbits[i, 0, :] # x-coordinate
        y = um_orbits[i, 1, :] # y-coordinate
        plt.plot(x, y, lw=1.0)

```

```

# Mark the reference point  $X_*$ 
plt.plot(X_star[0], X_star[1], "ro", ms=6, label=r"$X_\ast$")

plt.xlabel("x")
plt.ylabel("y")
plt.title(title)
plt.axis("equal")
plt.grid(True, alpha=0.3)
plt.legend()
plt.tight_layout()
plt.show()

```

```

[28]: import numpy as np
import matplotlib.pyplot as plt

# Fast RK4 integrator for ONE 6D orbit (no SciPy, no JAX jit)
def rk4_orbit_single(X0, dt=1e-2, n_steps=2000):
    """
    Integrate a single trajectory using a fixed-step RK4 method.

    Parameters
    -----
    X0 : array_like, shape (6,)
        Initial condition [x, y, z, vx, vy, vz].
    dt : float
        Time step.
    n_steps : int
        Number of steps.

    Returns
    -----
    traj : ndarray, shape (n_steps+1, 6)
        Trajectory  $X(t_n)$ ,  $n = 0, \dots, n\_steps$ .
    """
    X0 = np.asarray(X0, dtype=float)
    traj = np.zeros((n_steps + 1, 6), dtype=float)
    traj[0] = X0

    # keep the state as a jnp array when calling vectorField
    x = jnp.array(X0)

    for n in range(n_steps):
        k1 = vectorField(x)
        k2 = vectorField(x + 0.5 * dt * k1)
        k3 = vectorField(x + 0.5 * dt * k2)
        k4 = vectorField(x + dt * k3)

```

```

    x = x + dt * (k1 + 2*k2 + 2*k3 + k4) / 6.0
    traj[n + 1] = np.array(x, dtype=float)

return traj

# Nonlinear unstable manifold using pushed-forward perturbations
def compute_unstable_manifold_rk4(orbit,
    perts,
    orbit_index=0,
    t0_index=0,
    eps=1e-3,
    n_branches=11,
    dt=1e-2,
    n_steps=2000):
    """
    Nonlinear (refined) approximation of the local unstable manifold
    using pushed-forward perturbations along the leading Lyapunov vector,
    integrated with a fast fixed-step RK4 scheme.

    Parameters
    -----
    orbit : array, shape (n_time, 6, batch_size)
        Orbit array returned by `tangent` (WITHOUT transpose).
        orbit[t, :, i] is  $X^{(i)}(t)$ .
    perts : array, shape (n_time, 6, 6, batch_size)
        Lyapunov vectors array returned by `tangent` (WITHOUT transpose).
        perts[t, :, 0, i] is the leading LV  $e_1$  at time index t, orbit i.
    orbit_index : int
        Which orbit in the batch to use as the reference orbit.
    t0_index : int
        Time index along that orbit used as the reference point  $X_* = X(t_0)$ .
    eps : float
        Max size of the perturbation s in  $X_* + s e_1(X_*)$ .
    n_branches : int
        Number of different s-values in  $[-eps, eps]$ .
    dt : float
        RK4 time step.
    n_steps : int
        Number of RK4 steps for each branch.

    Returns
    -----
    um_orbits : ndarray, shape (n_branches, 6, n_steps+1)
        Nonlinear trajectories  $X_s(t_n)$  for each branch.
    X_star : ndarray, shape (6,)

```

```

    Reference point X_*.
    e1_star : ndarray, shape (6,)
    Leading Lyapunov vector at X_*.
    t_grid : ndarray, shape (n_steps+1,)
    Time grid used for the RK4 integration.
    """
    # Reference point X_* and leading LV e_1(X_*)
    X_star = np.array(orbit[t0_index, :, orbit_index], dtype=float) # (6,)
    e1_star = np.array(perts[t0_index, :, 0, orbit_index], dtype=float)

    # Parameter values s in [-eps, eps]
    s_vals = np.linspace(-eps, eps, n_branches)

    um_orbits = [] # list of (n_steps+1, 6) arrays

    for s in s_vals:
        X0_s = X_star + s * e1_star
        traj_s = rk4_orbit_single(X0_s, dt=dt, n_steps=n_steps)
        # transpose to shape (6, n_steps+1) for convenience
        um_orbits.append(traj_s.T)

    um_orbits = np.stack(um_orbits, axis=0) # (n_branches, 6, n_steps+1)
    t_grid = np.linspace(0.0, dt * n_steps, n_steps + 1)

    return um_orbits, X_star, e1_star, t_grid

def plot_unstable_manifold(um_orbits, X_star,
                           title="Local unstable manifold (x-y projection)":
    """
    Plot the x-y projection of the nonlinear unstable manifold branches.

    Parameters
    -----
    um_orbits : ndarray, shape (n_branches, 6, n_t)
        Output of compute_unstable_manifold_rk4.
    X_star : ndarray, shape (6,)
        Reference point X_* used to seed the manifold.
    """
    n_branches, _, n_t = um_orbits.shape

    plt.figure(figsize=(8, 6))
    for i in range(n_branches):
        x = um_orbits[i, 0, :]
        y = um_orbits[i, 1, :]
        plt.plot(x, y, lw=1.0)

```

```

plt.plot(X_star[0], X_star[1], "ro", ms=6, label=r"$X_{\ast}$")
plt.xlabel("x")
plt.ylabel("y")
plt.title(title)
plt.axis("equal")
plt.grid(True, alpha=0.3)
plt.legend()
plt.tight_layout()
plt.show()

```

```

[29]: if __name__=="__main__":
    batch_size = 100
    rng = jax.random.PRNGKey(42)
    L2_point = jnp.array([1 + mu2, 0, 0, 0, 0, 0]) # L2 Lagrange point
    perturbation_scale = 1e-3
    perturbations = jax.random.normal(rng, (batch_size, 6)) * perturbation_scale
    state = L2_point + perturbations

    le, orbit, perts = tangent(state, rng)

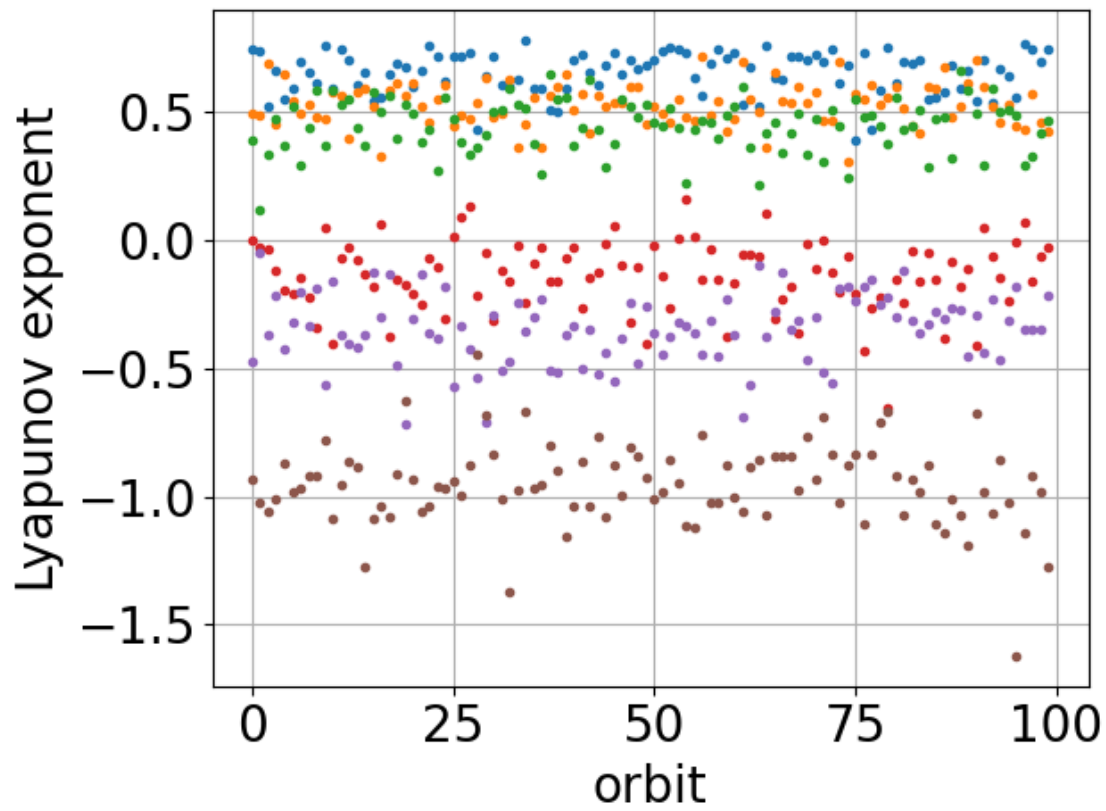
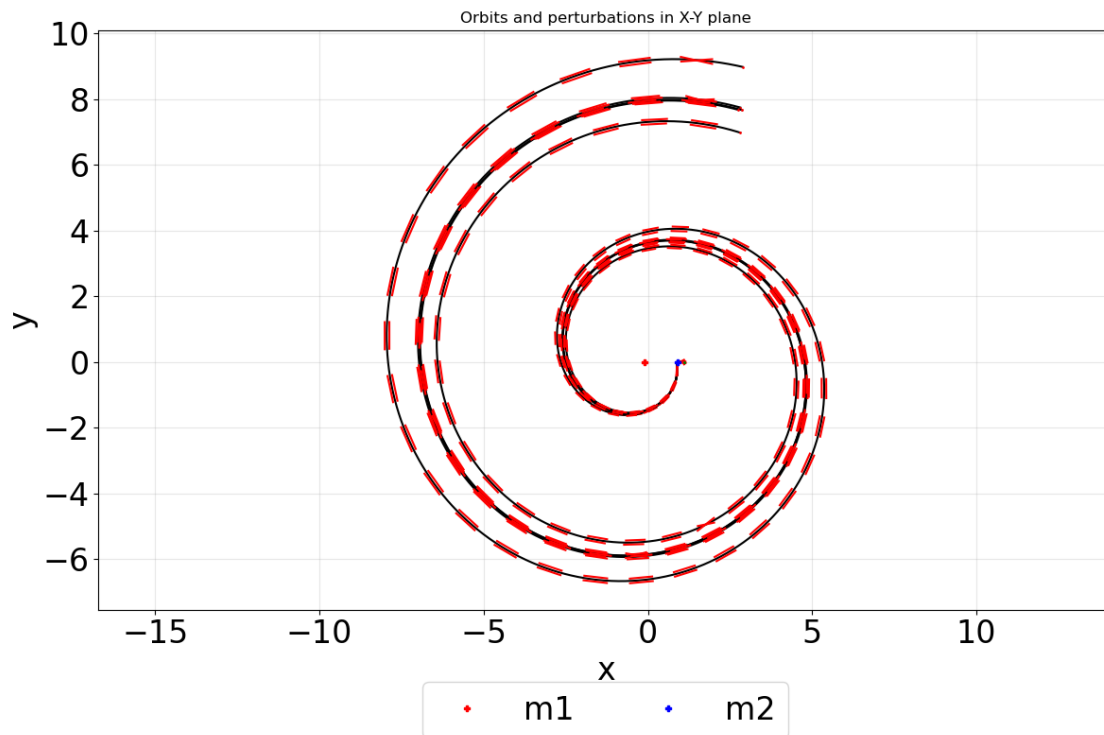
    plot_lvs_xy(orbit.T, perts.T)
    plot_les(le)
    # -----
    # Nonlinear unstable manifold seeded at X_* on orbit 0
    # -----
    orbit_index = 0 # which orbit in the batch
    t0_index = 0 # time index along that orbit (use the initial point)

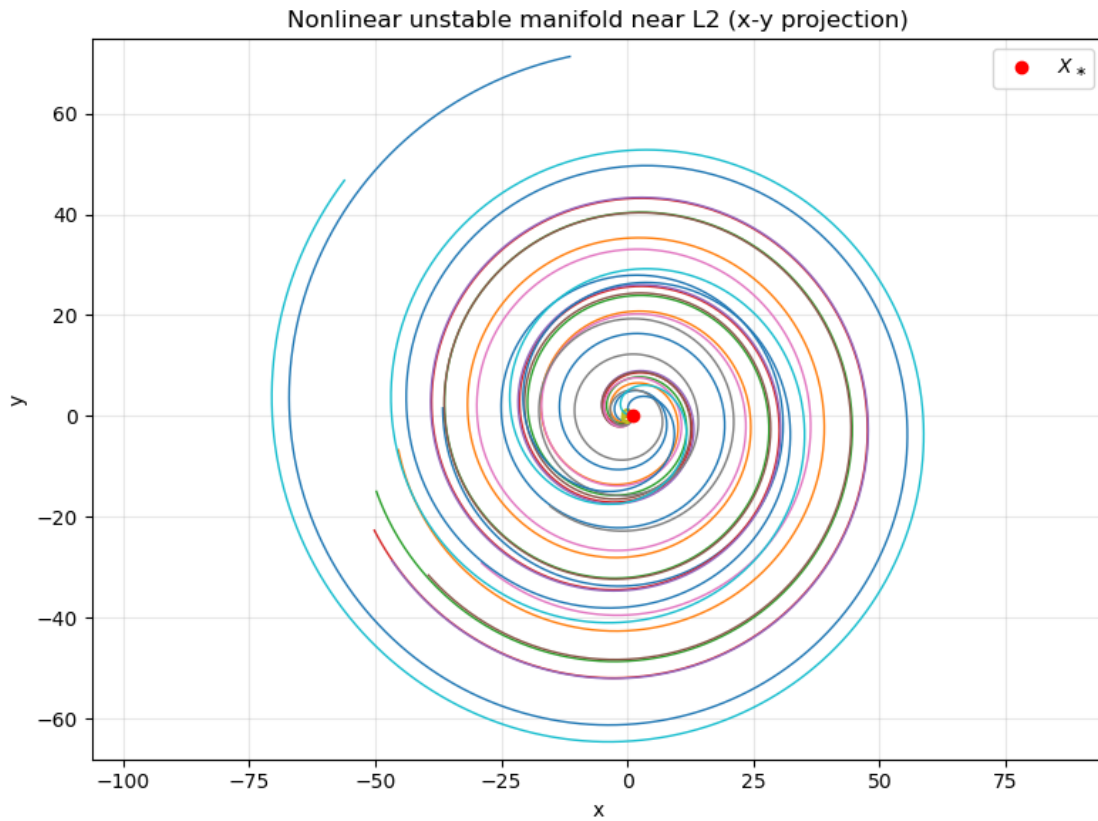
    um_orbits, X_star, e1_star, t_grid = compute_unstable_manifold_rk4(
        orbit, perts,
        orbit_index=orbit_index,
        t0_index=t0_index,
        eps=1e-3,
        n_branches=11,
        dt=1e-2,
        n_steps=2000,
    )

    plot_unstable_manifold(
        um_orbits,
        X_star,
        title="Nonlinear unstable manifold near L2 (x-y projection)"
    )

```

100%| | 10000/10000 [20:20<00:00, 8.19it/s]





```
[33]: import jax
import jax.numpy as jnp
import numpy as np
import matplotlib.pyplot as plt

#
# Divergence of V via automatic differentiation

# We assume vectorField(X) is already defined and returns a 6D JAX array.
# X has shape (6,) -> vectorField(X) has shape (6,)

# jacobian_V(X) : 6x6 Jacobian matrix DV(X)
jacobian_V = jax.jacfwd(vectorField)

def divergence_V_single(X):
    """
    Compute the divergence  $\nabla \cdot V$  at a single point  $X \in \mathbb{R}^6$ 
    using automatic differentiation.
```

```

Parameters
-----
X : array_like, shape (6,)
    Phase-space point.

Returns
-----
divV : float
    Divergence  $\nabla \cdot V(X)$ .
    """
X = jnp.array(X)
DV = jacobian_V(X)           # shape (6, 6)
divV = jnp.trace(DV)         # sum of diagonal entries
return float(divV)

def divergence_V_batch(X_batch):
    """
    Compute divergence  $\nabla \cdot V$  at a batch of points.

    Parameters
    -----
    X_batch : array_like, shape (N, 6)

    Returns
    -----
    divs : np.ndarray, shape (N,)
        Divergence values at each point.
    """
    X_batch = jnp.array(X_batch)           # (N, 6)

    # vmap over the first axis: X_batch[i,:] -> divergence_V_single
    DV_batch = jax.vmap(jacobian_V)(X_batch) # shape (N, 6, 6)
    divs = jnp.trace(DV_batch, axis1=1, axis2=2) # shape (N,)

    return np.array(divs, dtype=float)

def check_divergence_statistics(
    n_samples=200,
    perturbation_scale=1e-2,
    use_L2_center=True
):
    """
    Sample points near L2 (or near the origin) and compute  $\nabla \cdot V(X)$ .
    Print statistics and plot a histogram.

```

```

Parameters
-----
n_samples : int
    Number of sample points in phase space.
perturbation_scale : float
    Std dev of the Gaussian perturbation around the center point.
use_L2_center : bool
    If True, center samples around the approximate L2 point
     $X_c = (1 + \mu_2, 0, 0, 0, 0, 0)$ .
    If False, center around the origin.
"""
# Choose a center point in phase space
if use_L2_center:
    X_center = jnp.array([1.0 + mu2, 0., 0., 0., 0., 0.])
else:
    X_center = jnp.zeros(6)

rng = np.random.default_rng(0)
perturb = perturbation_scale * rng.standard_normal((n_samples, 6))
X_batch = np.array(X_center) + perturb

divs = divergence_V_batch(X_batch)

print("=== Divergence test for V (original flow) ===")
print(f"Number of samples: {n_samples}")
print(f"Mean(·V) = {divs.mean(): .3e}")
print(f"Std(·V) = {divs.std(): .3e}")
print(f"Min(·V) = {divs.min(): .3e}")
print(f"Max(·V) = {divs.max(): .3e}")

plt.figure(figsize=(6,4))
plt.hist(divs, bins=20, edgecolor="k", alpha=0.7)
plt.axvline(0.0, color="r", linestyle="--", label="0")
plt.xlabel(r" $\nabla \cdot V(X)$ ")
plt.ylabel("count")
plt.title("Histogram of divergence of V at sampled points")
plt.legend()
plt.tight_layout()
plt.show()

# Cloud-of-points test: volume of covariance ellipsoid
i
def rk4_step(x, dt):
    """
    One RK4 step for the original CR3BP flow  $\dot{X} = V(X)$ .

```

```

Parameters
-----
x : jnp.ndarray, shape (6,)
dt : float

Returns
-----
x_next : jnp.ndarray, shape (6,)
"""
k1 = vectorField(x)
k2 = vectorField(x + 0.5 * dt * k1)
k3 = vectorField(x + 0.5 * dt * k2)
k4 = vectorField(x + dt * k3)
return x + dt * (k1 + 2*k2 + 2*k3 + k4) / 6.0

def rk4_orbit_cloud(X0_cloud, dt=1e-2, n_steps=1000):
    """
    Integrate a cloud of points forward in time with RK4.

    Parameters
    -----
    X0_cloud : array_like, shape (N, 6)
        Initial conditions for each of N particles.
    dt : float
        Time step.
    n_steps : int
        Number of RK4 steps.

    Returns
    -----
    X_cloud : np.ndarray, shape (n_steps+1, N, 6)
        Trajectory of the cloud. X_cloud[n,i,:] is the state of particle i at_
↪ step n.
    """
    X = jnp.array(X0_cloud)    # (N, 6)
    N = X.shape[0]

    X_hist = np.zeros((n_steps+1, N, 6), dtype=float)
    X_hist[0] = np.array(X, dtype=float)

    for n in range(n_steps):
        # vmap RK4 over all particles
        X = jax.vmap(rk4_step, in_axes=(0, None))(X, dt)
        X_hist[n+1] = np.array(X, dtype=float)

    return X_hist

```

```

def covariance_volume(points):
    """
    Volume proxy for a set of points in  $R^6$ :  $\sqrt{\det(\text{covariance matrix})}$ .

    Parameters
    -----
    points : array_like, shape (N, 6)

    Returns
    -----
    vol : float
        Proportional to the volume of the covariance ellipsoid.
    """
    points = np.asarray(points)
    mean = points.mean(axis=0, keepdims=True)
    Y = points - mean
    N = points.shape[0]
    # sample covariance matrix
    C = (Y.T @ Y) / (N - 1)          # shape (6, 6)
    detC = np.linalg.det(C)
    if detC <= 0:
        # numerical noise or degenerate cloud
        return 0.0
    return float(np.sqrt(detC))

def check_volume_preservation_cloud(
    n_points=200,
    eps=1e-3,
    dt=1e-2,
    n_steps=1000,
    use_L2_center=True
):
    """
    Evolve a cloud of initial conditions under the flow and monitor the
    volume of their covariance ellipsoid.

    Parameters
    -----
    n_points : int
        Number of particles in the cloud (should be 6).
    eps : float
        Scale of the initial Gaussian perturbations.
    dt : float
        RK4 time step.

```

```

n_steps : int
    Number of RK4 steps.
use_L2_center : bool
    If True, center cloud near approximate L2 point,
    otherwise near the origin.
"""
if use_L2_center:
    X_center = np.array([1.0 + float(mu2), 0., 0., 0., 0., 0.])
else:
    X_center = np.zeros(6)

rng = np.random.default_rng(1)
perturb = eps * rng.standard_normal((n_points, 6))
X0_cloud = X_center + perturb    # initial cloud (N, 6)

# Integrate cloud under the original vector field
X_hist = rk4_orbit_cloud(X0_cloud, dt=dt, n_steps=n_steps)

# Compute volume proxy at each time step
vols = np.zeros(n_steps+1)
for n in range(n_steps+1):
    vols[n] = covariance_volume(X_hist[n])

t_grid = dt * np.arange(n_steps+1)

print("=== Cloud-of-points volume test ===")
print(f"N points      = {n_points}")
print(f"eps            = {eps}")
print(f"dt              = {dt}")
print(f"final time T     = {t_grid[-1]:.3f}")
print(f"vol(0)           = {vols[0]:.6e}")
print(f"vol(T)           = {vols[-1]:.6e}")
print(f"ratio vol(T)/vol(0) = {vols[-1]/vols[0]:.6f}")

plt.figure(figsize=(6,4))
plt.plot(t_grid, vols, "-o", ms=3)
plt.xlabel("time")
plt.ylabel("volume proxy   $\sqrt{\det \Sigma(t)}$ ")
plt.title("Evolution of cloud volume under the flow  $\dot{X} = V(X)$ ")
plt.grid(True)
plt.tight_layout()
plt.show()

```

Cell In[33], line 108

```

i
^

```

SyntaxError: invalid character 'i' (U+00A1)

```
[34]: # Divergence-based check (should be very close to 0 everywhere)
check_divergence_statistics(n_samples=200, perturbation_scale=1e-2)

# Cloud-of-points volume check (volume should stay ~constant)
check_volume_preservation_cloud(
    n_points=200,
    eps=1e-3,
    dt=1e-2,
    n_steps=1000,
    use_L2_center=True
)
```

=== Divergence test for V (original flow) ===

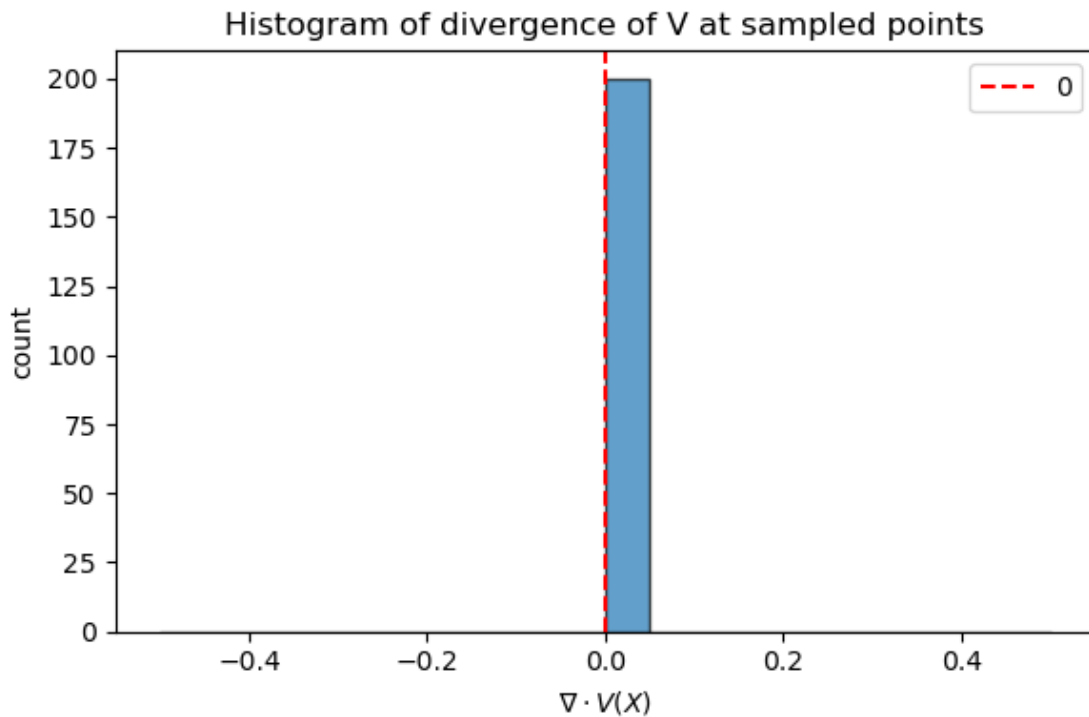
Number of samples: 200

Mean(·V) = 0.000e+00

Std(·V) = 0.000e+00

Min(·V) = 0.000e+00

Max(·V) = 0.000e+00



=== Cloud-of-points volume test ===

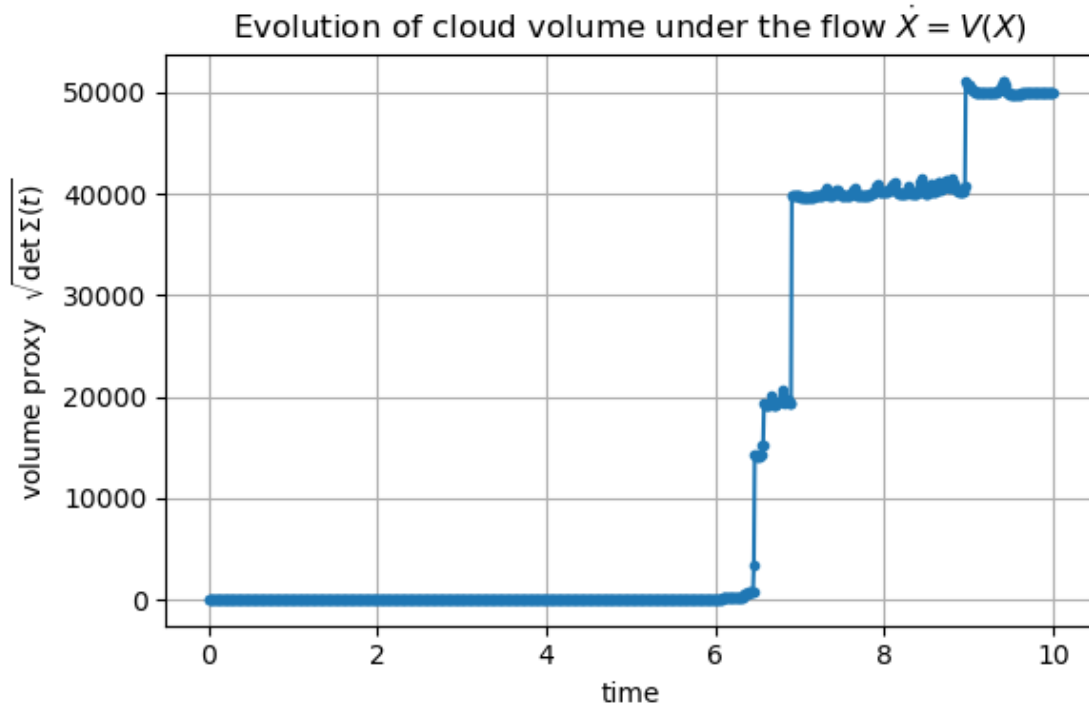
N points = 200

eps = 0.001

```

dt                = 0.01
final time T      = 10.000
vol(0)            = 9.185126e-19
vol(T)            = 4.998535e+04
ratio vol(T)/vol(0) = 54419881229363961659392.000000

```



```

[35]: import numpy as np

def lyapunov_targeting(X_target,
                       orbit,
                       perts,
                       le,
                       orbit_index=0,
                       time_index=200,
                       dt_tangent=1e-3,
                       use_stable_only=False,
                       verbose=True):

    # Convert to NumPy for linear algebra
    orbit_np = np.array(orbit)
    perts_np = np.array(perts)
    le_np = np.array(le)

    # Reference orbit and time

```



```

X0_ref = orbit_np[0, :, orbit_index]          #  $X_*(0)$ 
Xt_ref = orbit_np[time_index, :, orbit_index] #  $X_*(t)$ 
t = time_index * dt_tangent

# Q matrices of Lyapunov vectors at t=0 and t
# perts[t,:,:,:] stores  $Q^T$ , so we transpose back
U0 = perts_np[0, :, :, orbit_index]          # shape (6,6) =  $Q0^T$ 
Ut = perts_np[time_index, :, :, orbit_index] # shape (6,6) =  $Qt^T$ 
Q0 = U0.T                                     # columns ~  $e_i(0)$ 
Qt = Ut.T                                     # columns ~  $e_i(t)$ 

# Re-orthonormalize to be safe
Q0, _ = np.linalg.qr(Q0)
Qt, _ = np.linalg.qr(Qt)

# Decompose the target offset in the LV basis at time t
Delta_t = np.asarray(X_target) - Xt_ref        # desired delta at time t
b = Qt.T @ Delta_t                            # coefficients in LV
↪ basis at t

lambdas = le_np[orbit_index, :]                #  $\lambda_i$ 

# Optionally restrict to stable directions
if use_stable_only:
    mask = lambdas < 0.0
else:
    mask = np.ones_like(lambdas, dtype=bool)

# Compute initial coefficients  $a_i = b_i * \exp(-\lambda_i t)$ 
a = np.zeros_like(b)
for i in range(6):
    if not mask[i]:
        continue
    a[i] = b[i] * np.exp(-lambdas[i] * t)

# Initial perturbation in physical coordinates:  $X(0) = \sum a_i e_i(0)$ 
delta_X0 = Q0 @ a
X0_new = X0_ref + delta_X0

if verbose:
    print("=== Lyapunov targeting summary ===")
    print(f"orbit_index = {orbit_index}, time_index = {time_index}, t = {t:.3e}")
    print("Reference  $X_*(0)$  :", X0_ref)
    print("Reference  $X_*(t)$  :", Xt_ref)
    print("Target  $X_{\text{target}}$ :", np.asarray(X_target))
    print("Initial guess  $X0_{\text{new}}$ :", X0_new)

```

```

    print("||delta X(0)|| =", np.linalg.norm(delta_X0))
    if use_stable_only:
        print("Used only stable directions ( $\lambda_i < 0$ ).")
    else:
        print("Used all Lyapunov directions.")

    return X0_new

# Choose which orbit and time we want to target
orbit_index = 0
time_index = 200          # t = 0.2 with dt_tangent = 1e-3
dt_tangent = 1e-3

orbit_np = np.array(orbit)
Xt_ref = orbit_np[time_index, :, orbit_index]

# Construct a nearby target point X_tar (for demonstration)
X_target = Xt_ref.copy()
X_target[0] += 1e-3      # small shift in x
X_target[1] -= 5e-4      # small shift in y

# Use Lyapunov targeting to get an initial condition
X0_guess = lyapunov_targeting(
    X_target,
    orbit,
    perts,
    le,
    orbit_index=orbit_index,
    time_index=time_index,
    dt_tangent=dt_tangent,
    use_stable_only=False,
    verbose=True,
)

# Integrate from this initial condition to time t and check the miss distance
T = time_index * dt_tangent
sol = integrate_system(X0_guess, (0.0, T), t_eval=[T])
X_at_T = sol.y[:, -1]

print("\nDistance between X(T; X0_guess) and X_target:")
print("|| X(T) - X_target || =", np.linalg.norm(X_at_T - X_target))

=== Lyapunov targeting summary ===
orbit_index = 0, time_index = 200, t = 2.000e-01
Reference X_*(0) : [ 1.0994626e+00 -1.0913785e-03  6.2072260e-04 -2.4773991e-03
 1.2239548e-03 -2.9051682e-04]
Reference X_*(t) : [ 1.0546721e+00  4.9563125e-03  3.9586858e-04 -4.9440885e-01

```

```
8.7966949e-02 -2.0498885e-03]
Target X_target: [ 1.0556722e+00  4.4563124e-03  3.9586858e-04 -4.9440885e-01
8.7966949e-02 -2.0498885e-03]
Initial guess X0_new: [ 1.1000893e+00 -1.2417911e-03  1.6816074e-03
-2.4365671e-03
1.5896480e-03 -2.5683566e-04]
||delta X(0)|| = 0.0012951343
Used all Lyapunov directions.

Distance between X(T; X0_guess) and X_target:
|| X(T) - X_target || = 0.005131483339738632
```

[]: